

Prototype Design Pattern — Interview & Engineering Handbook

PHP 8.3 · Laravel · Node.js · Saudi Arabia | Dubai/UAE | Malaysia | India
Tier-2 | India Tier-1/60LPA+ (Bilingual English + Hindi)

Interview Prep Reference · Updated August 2026

Prototype Design Pattern

A note on length before you start: Prototype is a **Low-frequency** pattern per the research in `design-patterns-frequency-guide-expanded.md` — it is not a headline LLD round anywhere in your five target markets. This handbook is deliberately shorter than what you'd want for Strategy, Factory, Observer, or Singleton (your Very High priority patterns). Don't mistake the brevity for incompleteness — it's calibrated to how much interview time this pattern actually earns. Spend your remaining prep budget on the Very High patterns.

शुरू करने से पहले लंबाई पर एक टिप्पणी: `design-patterns-frequency-guide-expanded.md` के शोध के अनुसार Prototype एक **कम-फ्रीक्वेंसी** पैटर्न है — यह आपके पाँच लक्षित बाज़ारों में कहीं भी एक हेडलाइन LLD राउंड नहीं है। यह हैंडबुक जानबूझकर Strategy, Factory, Observer, या Singleton (आपके Very High प्राथमिकता वाले पैटर्न्स) से छोटी है। संक्षिप्तता को अधूरापन न समझें — यह इस बात पर कैलिब्रेटेड है कि यह पैटर्न असल में कितना इंटरव्यू समय कमाता है। अपना बाक़ी तैयारी बजट Very High पैटर्न्स पर खर्च करें।

Companion file: all runnable code referenced here lives in `Prototype.php`, same folder. This document is theory and rehearsal; the code file is the lab.

साथी फ़ाइल: यहाँ रेफ़र किया गया सारा रनेबल कोड `Prototype.php` में है, उसी फ़ोल्डर में। यह दस्तावेज़ थ्योरी और रिहर्सल है; कोड फ़ाइल प्रयोगशाला है।

⚡ FAST TRACK — read this every time you revisit this pattern

Part 1 — 60-Second Recall Card

Category	Creational (GoF)
One-liner	Create new objects by cloning an existing, fully-configured instance instead of building from scratch
Core mechanism (PHP)	<code>clone</code> keyword + <code>__clone()</code> magic method
Problem it solves	Repeated expensive object construction (I/O, parsing, crypto)
Trigger phrase	"Expensive setup," "same object thousands of times," "template + small variation"
Anti-trigger	Cheap objects; genuinely varying types (→ Factory); session/auth/payment-transaction state (never clone this)
#1 interview follow-up	Shallow vs. deep copy — PHP's <code>clone</code> is shallow by default; nested objects are shared references until <code>__clone()</code> says otherwise
Closest confused patterns	Factory (creates fresh, doesn't need an existing instance), Builder (step-by-step, no existing instance), Singleton (exactly one instance, ever — actively blocks cloning)
Memory hook	<i>Prototype = rubber stamp.</i> Carving the stamp (construction) happens once; stamping paper (cloning) is cheap and repeated.

हिंदी अनुवाद / Hindi Translation:

श्रेणी	क्रिएशनल (GoF)
एक-पंक्ति सार	शुरू से बनाने के बजाय एक मौजूदा, पूरी तरह-कॉन्फिगर्ड इंस्टेंस को क्लोन करके नए ऑब्जेक्ट्स बनाएँ
मुख्य तंत्र (PHP)	<code>clone</code> कीवर्ड + <code>__clone()</code> मैजिक मेथड
हल की गई समस्या	दोहराया गया महँगा ऑब्जेक्ट निर्माण (I/O, पार्सिंग, क्रिप्टो)
ट्रिगर वाक्यांश	"महँगा सेटअप," "हज़ारों बार एक जैसा ऑब्जेक्ट," "टेम्पलेट + छोटा बदलाव"
एंटी-ट्रिगर	सस्ते ऑब्जेक्ट्स; सचमुच बदलती टाइप्स (→ Factory); session/auth/payment-transaction स्थिति (इसे कभी क्लोन न करें)
#1 इंटरव्यू फ़ॉलो-अप	Shallow बनाम deep कॉपी — PHP का <code>clone</code> डिफ़ॉल्ट रूप से shallow है; nested ऑब्जेक्ट्स तब तक साझा रेफ़रेंस हैं जब तक <code>__clone()</code> कुछ और न कहे
सबसे मिलते-जुलते भ्रामक पैटर्न्स	Factory (ताज़ा बनाता है, मौजूदा इंस्टेंस की ज़रूरत नहीं), Builder (चरण-दर-चरण, कोई मौजूदा इंस्टेंस नहीं), Singleton (हमेशा ठीक एक इंस्टेंस — cloning को सक्रिय रूप से ब्लॉक करता है)

याद रखने की तरकीब	<i>Prototype</i> = रबर स्टैम्प। स्टैम्प तराशना (निर्माण) एक बार होता है; कागज़ पर स्टैम्प लगाना (cloning) सस्ता और दोहराया जाने वाला है।
-------------------	--

Part 2 — Market Calibration

Pulled directly from `design-patterns-frequency-guide-expanded.md` — no claim here that isn't traceable to that research.

सीधे `design-patterns-frequency-guide-expanded.md` से लिया गया — यहाँ कोई दावा नहीं जो उस शोध तक ट्रेस न हो सके।

Market	Confirmed as a named/ standalone topic?	Realistic role
Saudi Arabia	No	Not observed in any researched company's LLD round. If it appears, expect it buried inside a caching or template-duplication discussion (e.g. HungerStation's "architecture and design patterns" round), not as its own prompt.
Dubai/UAE	No	Same — not a named topic at Careem, Property Finder, Talabat, or any other researched UAE company. Possible follow-up inside a caching-layer or templating discussion.
Malaysia	No	Not observed. Malaysia's confirmed emphasis is Singleton (live-coded) and Strategy — Prototype has no footprint in the research.
India Tier-2	No	Not observed as a standalone topic across Razorpay, PhonePe, Swiggy, CRED, Freshworks, or the other 24 researched companies. The closest adjacent confirmed topics are LRU Cache design (PolicyBazaar, CRED) and multilevel caching (PhonePe).
India Tier-1/60LPA+	No	Not observed across Amazon, Google, Microsoft, Atlassian, Rippling, or the other 22 researched companies either.

हिंदी सार (बाज़ार दर हिसाब से): Saudi Arabia — नहीं, किसी भी शोधित कंपनी के LLD राउंड में नहीं देखा गया। अगर आता है, तो संभवतः एक caching या template-duplication चर्चा के अंदर दबा हुआ, अपने खुद के प्रॉम्प्ट के तौर पर नहीं। Dubai/UAE — वही, Careem, Property Finder, Talabat, या किसी अन्य शोधित UAE कंपनी में नामित विषय नहीं। Malaysia — नहीं देखा गया; Malaysia का पुष्ट ज़ोर Singleton (लाइव-कोडेड) और Strategy है। India Tier-2 — Razorpay, PhonePe, Swiggy, CRED, Freshworks, या अन्य 24 शोधित कंपनियों में स्वतंत्र विषय के तौर पर नहीं देखा गया; सबसे क़रीबी संबंधित पुष्ट विषय LRU Cache डिज़ाइन और multilevel caching हैं। India Tier-1/60LPA+ — Amazon, Google, Microsoft, Atlassian, Rippling, या अन्य 22 शोधित कंपनियों में भी नहीं देखा गया।

Honest bottom line: across ~145 companies researched in five markets, Prototype was not confirmed as a named interview topic anywhere. That doesn't mean skip it — it means: (1) know it well enough to recognize and name it *if* a caching/templating/object-duplication LLD

problem naturally calls for it, (2) know the shallow-vs-deep-copy mechanics cold, because that specific gotcha is genuinely general-purpose PHP/OOP knowledge, and (3) don't over-invest prep time here relative to your Very High priority patterns.

ईमानदार निचोड़: पाँच बाज़ारों में शोधित ~145 कंपनियों में, Prototype कहीं भी एक नामित इंटरव्यू विषय के तौर पर पुष्ट नहीं हुआ। इसका मतलब इसे छोड़ना नहीं है — इसका मतलब है: (1) इसे इतनी अच्छी तरह जानें कि अगर एक caching/templating/object-duplication LLD समस्या स्वाभाविक रूप से इसकी माँग करे तो इसे पहचान और नाम दे सकें (2) shallow-बनाम-deep-copy की कार्यप्रणाली को रट लें, क्योंकि वह खास गड़बड़ी सचमुच सामान्य-प्रयोजन PHP/OOP ज्ञान है, और (3) अपने Very High प्राथमिकता वाले पैटर्न्स के मुकाबले यहाँ तैयारी के समय में अति-निवेश न करें।

Part 3 — Recognition, Decision Tree & When NOT to Use

Requirement phrases that signal Prototype: - "Object setup/initialization is expensive or slow" - "We create the same kind of object thousands of times per batch/request" - "Most fields are identical; only two or three vary per instance" - "We already have a fully-configured object — we just need another one like it"

Prototype का संकेत देने वाले शब्द/वाक्यांश: - "ऑब्जेक्ट सेटअप/इनिशियलाइज़ेशन महँगा या धीमा है" - "हम हर बैच/रिक्वेस्ट में हज़ारों बार एक जैसा ऑब्जेक्ट बनाते हैं" - "ज़्यादातर फ़िल्ड्स एक जैसी हैं; प्रति-इंस्टेंस सिर्फ़ दो या तीन बदलती हैं" - "हमारे पास पहले से एक पूरी तरह-कॉन्फ़िगर्ड ऑब्जेक्ट है — हमें बस उसके जैसा एक और चाहिए"

Decision tree:

```
Is construction meaningfully expensive (I/O, crypto, parsing, computation)?
├─ No → just use `new`. Stop.
├─ Yes → Are you creating MANY structurally-similar instances from shared baseline state?
│   └─ No (types genuinely differ) → Factory problem, not Prototype. Stop.
│   └─ Yes → Does the object graph contain mutable nested objects (not just scalars)?
│       └─ No → default shallow clone is fine. Done.
│       └─ Yes → clone WITH a correctly-cascading __clone() override for every mutable nested object. Done.
```

निर्णय वृक्ष — हिंदी सार: पहले पूछें: क्या निर्माण सार्थक रूप से महँगा है (I/O, क्रिप्टो, पार्सिंग)? अगर नहीं, बस `new` इस्तेमाल करें, रुकें। अगर हाँ, पूछें: क्या आप साझा आधारभूत स्थिति से कई संरचनात्मक रूप से समान इंस्टेंसेज़ बना रहे हैं? अगर नहीं (टाइप्स सचमुच अलग हैं), यह Factory समस्या है, Prototype नहीं, रुकें। अगर हाँ, पूछें: क्या ऑब्जेक्ट ग्राफ़ में परिवर्तनशील nested ऑब्जेक्ट्स हैं (सिर्फ़ स्केलर्स नहीं)? अगर नहीं, डिफ़ॉल्ट shallow clone ठीक है, पूरा हुआ। अगर हाँ, हर परिवर्तनशील nested ऑब्जेक्ट के लिए सही ढंग से कैस्केड होने वाले `__clone()` ओवरराइड के साथ क्लोन करें, पूरा हुआ।

When NOT to use it (say these unprompted — it's a strong signal): - The object is cheap to construct — cloning adds overhead and complexity for nothing. - Object types genuinely vary per request (PaymentMethod: CreditCard vs. PayPal vs. BankTransfer) — that's Factory's job. - **The object holds session, auth, or payment-transaction state.** Cloning risks silently duplicating a reference to sensitive shared state across independent operations — a correctness *and* security concern, not a style preference.

कब इसका इस्तेमाल न करें (बिना पूछे यह कहें — यह एक मज़बूत संकेत है): - ऑब्जेक्ट बनाना सस्ता है — cloning बिना किसी फ़ायदे के ओवरहेड और जटिलता जोड़ता है। - ऑब्जेक्ट टाइप्स सचमुच प्रति-रिक्वेस्ट बदलती हैं — यह Factory का काम है। - **ऑब्जेक्ट session, auth, या payment-transaction स्थिति रखता है।** Cloning स्वतंत्र ऑपरेशन्स में संवेदनशील साझा स्थिति के एक रेफ़रेंस को चुपचाप दोहराने का जोखिम रखता है — एक सटीकता और सुरक्षा चिंता, कोई शैली प्राथमिकता नहीं।

✓ **Before you move on:** (1) Name one requirement phrase that should make you think "Prototype." (2) Name one type of object you should never clone, and why.

✓ **आगे बढ़ने से पहले:** (1) एक ज़रूरत वाक्यांश बताएँ जो आपको "Prototype" सोचने पर मजबूर करे। (2) एक तरह का ऑब्जेक्ट बताएँ जिसे आपको कभी क्लोन नहीं करना चाहिए, और क्यों।

Part 4 — Cheat Sheet & Multi-Length Pitch

PHP mechanism	<code>clone</code> + optional <code>__clone()</code>
Default copy behavior	Shallow — scalars by value, objects by shared reference
<code>clone</code> calls <code>__construct()</code> ?	No — never
<code>__clone()</code> runs when?	After the shallow copy already exists, on the new object — a repair hook, not an interceptor
Cooperates with	Factory (often builds the initial prototype), Registry (keys multiple variants)
Structural fix beyond <code>__clone()</code> discipline	Make nested state immutable by default — removes the need for deep-copy logic entirely

हिंदी अनुवाद / Hindi Translation:

PHP तंत्र	<code>clone</code> + वैकल्पिक <code>__clone()</code>
डिफ़ॉल्ट कॉपी व्यवहार	Shallow — स्केलर्स वैल्यू से, ऑब्जेक्ट्स साझा रेफ़रेंस से
क्या <code>clone</code> , <code>__construct()</code> कॉल करता है?	नहीं — कभी नहीं
<code>__clone()</code> कब चलता है?	नए ऑब्जेक्ट पर, shallow कॉपी पहले से मौजूद होने के बाद — एक मरम्मत हुक, कोई इंटरसेप्टर नहीं
इनके साथ सहयोग करता है	Factory (अक्सर शुरुआती prototype बनाती है), Registry (कई वेरिएंट्स की-करती है)
<code>__clone()</code> अनुशासन से आगे संरचनात्मक फ़िक्स	nested स्थिति को डिफ़ॉल्ट रूप से अपरिवर्तनीय बनाएँ — deep-copy लॉजिक की ज़रूरत पूरी तरह हटाता है

30 seconds: "Prototype creates new objects by cloning an existing, fully-configured instance instead of constructing from scratch. It's used when construction is expensive — pay the setup

cost once, clone cheaply after. The catch: PHP's `clone` is shallow by default, so nested objects need an explicit deep-copy in `__clone()`, or you get shared-state bugs."

30 सेकंड: "Prototype शुरू से बनाने के बजाय एक मौजूदा, पूरी तरह-कॉन्फिगर्ड इंस्टेंस को क्लोन करके नए ऑब्जेक्ट्स बनाता है। इसका इस्तेमाल तब होता है जब निर्माण महंगा हो — सेटअप लागत एक बार चुकाएँ, बाद में सस्ते में क्लोन करें। पेच: PHP का `clone` डिफ़ॉल्ट रूप से shallow है, इसलिए nested ऑब्जेक्ट्स को `__clone()` में एक खुली deep-copy चाहिए, वरना आपको साझा-स्थिति बग्स मिलती हैं।"

1 minute: The 30-second version, plus: "A concrete example is invoice generation — branding, tax rules, and templates are expensive to assemble and identical across thousands of orders, so you build one prototype per tax jurisdiction, clone it per order, and overwrite only order-specific fields. The most common bug is forgetting a nested object like a tax profile is shared by reference after a shallow clone."

1 मिनट: 30-सेकंड वर्शन, साथ में: "एक ठोस उदाहरण है invoice जनरेशन — branding, tax नियम, और templates असेंबल करने में महँगे हैं और हज़ारों ऑर्डर्स में एक जैसे हैं, इसलिए आप प्रति-tax-क्षेत्राधिकार एक prototype बनाते हैं, प्रति-ऑर्डर इसे क्लोन करते हैं, और सिर्फ़ ऑर्डर-विशिष्ट फ़िल्ड्स ओवरराइट करते हैं। सबसे आम बग एक nested ऑब्जेक्ट जैसे tax profile को भूल जाना है, जो shallow clone के बाद रेफ़रेंस से साझा है।"

3 minutes: The 1-minute version, plus: PHP object internals in one sentence; the registry variant for multiple variants with event-driven invalidation; the explicit anti-pattern of cloning security/transaction-sensitive state; the concurrency nuance between PHP-FPM's per-request isolation and long-running processes like Swoole.

3 मिनट: 1-मिनट वर्शन, साथ में: एक वाक्य में PHP ऑब्जेक्ट आंतरिक कार्यप्रणाली; कई वेरिएंट्स के लिए registry वेरिएंट, event-driven invalidation के साथ; security/transaction-संवेदनशील स्थिति को क्लोन करने का खुला एंटी-पैटर्न; PHP-FPM के प्रति-रिक्वेस्ट अलगाव और Swoole जैसी लंबे समय तक चलने वाली प्रोसेसेज़ के बीच concurrency बारीकी।

10 minutes: The 3-minute version, plus: the full refactoring journey from a naive constructor to a tested, registry-backed production version (Part 19); the SOLID analysis; the comparison table against Factory/Builder/Singleton/Memento; the argument that immutability is the highest-leverage fix for the entire shallow-copy bug class; and the honest fact (Part 2) that this pattern doesn't headline any interview round in your target markets.

10 मिनट: 3-मिनट वर्शन, साथ में: एक सीधे-सादे कंस्ट्रक्टर से एक टेस्टेड, registry-समर्थित प्रोडक्शन वर्शन तक पूरी रीफ़ैक्टरिंग यात्रा (Part 19); SOLID विश्लेषण; Factory/Builder/Singleton/Memento के मुकाबले तुलना टेबल; यह तर्क कि immutability पूरी shallow-copy बग श्रेणी के लिए सबसे ऊँचे-लाभ वाला फ़िक्स है; और यह ईमानदार तथ्य (Part 2) कि यह पैटर्न आपके लक्षित बाज़ारों में किसी इंटरव्यू राउंड की हेडलाइन नहीं है।

Part 5 — Timed Mock Drill

Prompt (give yourself 45 minutes, no notes): *"We run a multi-region invoice service. Every invoice requires company branding, a tax profile (VAT/GST rate), a currency, and a PDF template — all identical for every order from the same country, loaded from a config service, a tax-rules service, and a template store respectively. During a flash sale we process 50,000*

orders in a short window and invoice generation is now the dominant contributor to processing latency. Design a solution."

प्रॉम्प्ट (खुद को 45 मिनट दें, कोई नोट्स नहीं): "हम एक मल्टी-रीजन *invoice* सेवा चलाते हैं। हर *invoice* को *company branding*, एक *tax profile*, एक करेंसी, और एक *PDF template* चाहिए — एक ही देश के हर ऑर्डर के लिए एक जैसे। एक फ्लैश सेल के दौरान हम एक छोटी विंडो में 50,000 ऑर्डर्स प्रोसेस करते हैं और *invoice* जनरेशन अब प्रोसेसिंग लेटेंसी का मुख्य योगदानकर्ता है। एक समाधान डिज़ाइन करें।"

Time-boxed sub-steps: - **0-5 min:** Clarify — is the bottleneck construction cost or something else? State your assumption explicitly if the interviewer doesn't answer. - **5-15 min:** Identify the expensive/cheap split. Sketch the class shape and the registry concept out loud before coding. - **15-30 min:** Write the `InvoicePrototype` class with a correct `__clone()`, and a keyed registry that never returns the stored master directly. - **30-40 min:** Handle the follow-up twist: "Tax rules just changed for one country mid-sale. How does this update without redeploying?" - **40-45 min:** State the trade-off you're making out loud, unprompted.

समय-सीमित उप-चरण: - **0-5 मिनट** — साफ़ करें — क्या बाधा निर्माण लागत है या कुछ और? अगर इंटरव्यूअर जवाब न दे तो अपनी धारणा खुलकर बताएँ। - **5-15 मिनट** — महँगे/सस्ते विभाजन की पहचान करें। कोड लिखने से पहले ज़ोर से क्लास आकार और registry विचार बताएँ। - **15-30 मिनट** — एक सही `__clone()` के साथ `InvoicePrototype` क्लास लिखें, और एक keyed registry जो कभी सीधे संग्रहीत मास्टर नहीं लौटाती। - **30-40 मिनट** — फ़ॉलो-अप मोड़ संभालें: "एक देश के लिए *sale* के बीच में *tax* नियम अभी बदले। यह बिना *redploy* किए कैसे अपडेट होता है?" - **40-45 मिनट** — बिना पूछे, ज़ोर से वह ट्रेड-ऑफ़ बताएँ जो आप बना रहे हैं।

Self-grading rubric — a bar-raiser-caliber interviewer is scoring: - Did you separate expensive/shared fields from cheap/instance-specific fields *before* writing code? - Did you get the registry to return a clone, never the master, without being prompted? - Did you preemptively mention the shallow-copy risk for the nested tax-profile object? - Did you handle the "tax rules changed mid-sale" twist with a structural answer? - Did you name the trade-off (registry staleness) unprompted?

स्वयं-ग्रेडिंग रूब्रिक: - क्या आपने कोड लिखने से पहले महँगे/साझा फ़िल्ड्स को सस्ते/इंस्टेंस-विशिष्ट फ़िल्ड्स से अलग किया? - क्या registry ने बिना पूछे एक क्लोन लौटाया, कभी मास्टर नहीं? - क्या आपने nested tax-profile ऑब्जेक्ट के लिए shallow-copy जोखिम पहले से बताया? - क्या आपने "sale के बीच में tax नियम बदले" मोड़ को एक संरचनात्मक जवाब से संभाला? - क्या आपने बिना पूछे ट्रेड-ऑफ़ (registry बासीपन) का नाम लिया?

✓ **Before you move on:** (1) What's the one design decision in this drill most candidates get right? (2) What's the one follow-up most candidates fumble?

✓ **आगे बढ़ने से पहले:** (1) इस ड्रिल में एक डिज़ाइन फ़ैसला क्या है जो ज़्यादातर उम्मीदवार सही करते हैं? (2) एक फ़ॉलो-अप क्या है जिसमें ज़्यादातर उम्मीदवार लड़खड़ाते हैं?

Part 6 — Pattern Recognition Drill

Scenario count here is intentionally short — Prototype's real footprint in your target markets is thin (Part 2), so a long drill would itself be padding. Five scenarios, covering the genuine confusion points with Factory, Builder, and Singleton.

यहाँ परिदृश्यों की संख्या जानबूझकर छोटी है — पाँच परिदृश्य, Factory, Builder, और Singleton के साथ असली भ्रम बिंदुओं को कवर करते हुए।

Scenario 1: "Design a notification system that sends templated emails/SMS in five languages. Branding and localized copy are loaded once; recipient and event data vary per send." →

Prototype (clone a per-locale template, overwrite recipient fields). *Not Factory* — the concrete "type" doesn't vary. *Not Builder* — no multi-step optional construction.

हिंदी: एक ऐसा नोटिफिकेशन सिस्टम डिज़ाइन करें जो पाँच भाषाओं में टेम्पलेटेड ईमेल/एसएमएस भेजता है। → **Prototype** (प्रति-locale एक टेम्पलेट क्लोन करें, प्राप्तकर्ता फ़िल्ड्स ओवरराइट करें)। *Factory* नहीं — कॉन्क्रीट "टाइप" बदलती नहीं। *Builder* नहीं — कोई बहु-चरणीय वैकल्पिक निर्माण नहीं।

Scenario 2: "Design a payment processing system supporting credit card, PayPal, and bank transfer, chosen at runtime based on the customer's selection." → **Factory**, not Prototype. The concrete type genuinely varies per request.

हिंदी: एक पेमेंट प्रोसेसिंग सिस्टम डिज़ाइन करें जो क्रेडिट कार्ड, PayPal, और बैंक ट्रांसफ़र सपोर्ट करे, रनटाइम पर चुना गया। → **Factory**, Prototype नहीं। कॉन्क्रीट टाइप सचमुच प्रति-रिक्वेस्ट बदलती है।

Scenario 3: "Design a report-generation service. Reports share a chart style and company logo; each report has a different date range and metric set." → **Prototype**. Same shape as invoice generation.

हिंदी: एक रिपोर्ट-जनरेशन सेवा डिज़ाइन करें। रिपोर्ट्स एक chart स्टाइल और कंपनी लोगो साझा करती हैं। → **Prototype**। Invoice जनरेशन जैसी ही बनावट।

Scenario 4: "Design a global app-config object that must exist exactly once for the lifetime of the application, and be accessible from anywhere." → **Singleton**, not Prototype — Singleton typically makes `__clone()` **private** specifically to *block* what Prototype relies on.

हिंदी: एक वैश्विक app-config ऑब्जेक्ट डिज़ाइन करें जिसे ऐप्लिकेशन के जीवनकाल के लिए ठीक एक बार अस्तित्व में होना चाहिए। → **Singleton**, Prototype नहीं — Singleton आमतौर पर `__clone()` को **प्राइवेट** बनाता है, खास तौर पर उसे ब्लॉक करने के लिए जिस पर Prototype निर्भर करता है।

Scenario 5: "Design an HTTP client builder that supports optionally setting headers, timeout, retries, and auth, in any combination, fluently." → **Builder**, not Prototype. Many optional, combinable construction parameters with no existing instance to copy from.

हिंदी: एक HTTP क्लाइंट बिल्डर डिज़ाइन करें जो वैकल्पिक रूप से headers, timeout, retries, और auth सेट करना सपोर्ट करे। → **Builder**, Prototype नहीं। कई वैकल्पिक, संयोजनीय निर्माण पैरामीटर्स, कॉपी करने के लिए कोई मौजूदा इंस्टेंस नहीं।

✓ **Before you move on:** Without looking up, state in one sentence each why scenario 2 isn't Prototype and why scenario 4 isn't Prototype.

✓ **आगे बढ़ने से पहले:** बिना देखे, एक-एक वाक्य में बताएँ कि परिदृश्य 2 Prototype क्यों नहीं है और परिदृश्य 4 Prototype क्यों नहीं है।

DEEP DIVE — read once, then use as reference

Path map: Fundamentals → Engineering Problem → Internals → Design (UML/Components) → Implementation (PHP/Laravel) → Production (scenarios + ADR) → Field Notes → Analogies/Architecture → SOLID/Performance/Concurrency → Trade-offs → Comparisons → Bugs/AI-review/Testing → Refactoring Journey → Practices/Mistakes/Traps → Interview Bank

पथ मानचित्र: बुनियाद → इंजीनियरिंग समस्या → आंतरिक कार्यप्रणाली → डिज़ाइन → इम्प्लीमेंटेशन → प्रोडक्शन → फ़िल्ड नोट्स → उपमाएँ/आर्किटेक्चर → SOLID/परफॉर्मेंस/Concurrency → ट्रेड-ऑफ़्स → तुलनाएँ → बग्स/AI-रिव्यू/टेस्टिंग → रीफैक्टिंग यात्रा → प्रैक्टिसेज़/गलतियाँ/जाल → इंटरव्यू बैंक

Part 7 — Fundamentals

Definition: Prototype is a Creational design pattern (GoF, 1994) that creates new objects by cloning an existing, fully-configured instance — the *prototype* — instead of constructing a fresh object from raw inputs every time.

परिभाषा: Prototype एक क्रिएशनल डिज़ाइन पैटर्न (GoF, 1994) है जो हर बार कच्चे इनपुट्स से एक ताज़ा ऑब्जेक्ट बनाने के बजाय, एक मौजूदा, पूरी तरह-कॉन्फिगर्ड इंस्टेंस — *prototype* — को क्लोन करके नए ऑब्जेक्ट्स बनाता है।

```
Traditional creation:  new ClassName(...args)  → constructor runs every time
Prototype creation:   clone $existingObject    → constructor runs ONCE, ever
```

The problem it solves: two related problems, one mechanism. First, expensive construction repeated many times. Second, needing "another object just like this one" without a clean Factory-style branching decision to encapsulate.

हल की गई समस्या: दो संबंधित समस्याएँ, एक तंत्र। पहला, बार-बार दोहराया गया महँगा निर्माण। दूसरा, एक साफ़ Factory-शैली की शाखाबद्ध फैसले को एनकैप्सुलेट किए बिना "इसके जैसा एक और ऑब्जेक्ट" चाहिए होना।

Beginner framing: think of a rubber stamp. Carving it takes effort; once it exists, stamping paper takes a second. You don't re-carve the stamp for every document — Prototype is "carve once, stamp many," applied to objects instead of ink.

शुरुआती स्तर की समझ: एक रबर स्टैम्प के बारे में सोचें। इसे तराशना मेहनत लेता है; एक बार बन जाने पर, कागज़ पर स्टैम्प लगाना एक सेकंड लेता है। आप हर दस्तावेज़ के लिए स्टैम्प फिर से नहीं तराशते — Prototype है "एक बार तराशो, कई बार स्टैम्प लगाओ," ऑब्जेक्ट्स पर लागू, स्याही के बजाय।

Senior/staff framing: Prototype decouples the *cost of object construction* from the *number of objects you need* — converting an $O(n)$ initialization cost into $O(1)$ initialization plus $O(n)$ cheap copies. The trade-off you're buying is the responsibility of correctly defining what "copy" means for that object graph.

सीनियर/स्टाफ़ स्तर की समझ: Prototype ऑब्जेक्ट निर्माण की लागत को आपको चाहिए ऑब्जेक्ट्स की संख्या से अलग करता है — एक $O(n)$ इनिशियलाइज़ेशन लागत को $O(1)$ इनिशियलाइज़ेशन प्लस $O(n)$ सस्ती कॉपियों में बदलते हुए। जो ट्रेड-ऑफ़ आप खरीद रहे हैं वह है यह सही ढंग से परिभाषित करने की ज़िम्मेदारी कि उस ऑब्जेक्ट ग्राफ़ के लिए "कॉपी" का क्या मतलब है।

✓ **Before you move on:** (1) State the GoF category and one other pattern in the same category. (2) In one sentence, what's the "senior" framing that a "mid" answer usually misses?

✓ **आगे बढ़ने से पहले:** (1) GoF श्रेणी और उसी श्रेणी का एक और पैटर्न बताएँ। (2) एक वाक्य में, "सीनियर" फ्रेमिंग क्या है जो एक "मिड" जवाब आमतौर पर चूक जाता है?

Part 8 — The Engineering Problem & Refactoring Trigger

What code looks like before this pattern: a constructor that mixes two categories of data that change at different rates — rarely-changing configuration and frequently-changing instance data — and re-fetches/re-parses the rarely-changing part on every single call.

यह पैटर्न लगाने से पहले कोड कैसा दिखता है: एक कंस्ट्रक्टर जो अलग-अलग दरों पर बदलने वाले डेटा की दो श्रेणियों को मिलाता है — शायद ही कभी बदलने वाला कॉन्फिगरेशन और बार-बार बदलने वाला इंस्टेंस डेटा — और हर एक कॉल पर शायद-ही-कभी-बदलने वाले हिस्से को फिर से लाता/पार्स करता है।

```
foreach ($orders as $order) {
    $invoice = new Invoice(
        companyName: fetchCompanyBranding(),           // network call, same every time
        taxProfile: fetchTaxProfile($order->region),    // network call, same every time
        pdfTemplate: loadPdfTemplate(),                 // disk/S3 read, same every time
        orderId: $order->id,                             // actually varies
        customerName: $order->customerName,             // actually varies
        amount: $order->amount,                         // actually varies
    );
}
```

Production-mindset questions — this is what actually separates a senior candidate's

answer here: - *What production problem actually forces this?* A specific, measurable symptom: p99 latency on invoice generation climbing during high-order-volume windows, traced to redundant calls to the branding/tax/template services. - *How would a senior engineer discover this before it's a crisis?* By noticing the shape of the cost in a profiler or APM trace. - *What metric would show it coming?* A rising ratio of "time spent in construction" to "time spent in domain logic" as order volume scales. - *What would a competent engineer try first, and why might they reject it?* Caching individual expensive fields — a reasonable intermediate step, but doesn't generalize and conflates caching with the class's responsibility.

प्रोडक्शन-सोच वाले सवाल — यहाँ यही चीज़ असल में एक सीनियर उम्मीदवार के जवाब को अलग करती है: - असल में कौन-सी प्रोडक्शन समस्या इसे मजबूर करती है? एक खास, मापने-योग्य लक्षण: उच्च-ऑर्डर-वॉल्यूम विंडोज़ के दौरान बढ़ती invoice जनरेशन p99 लेटेंसी। - एक सीनियर इंजीनियर संकट बनने से पहले इसे कैसे खोजेगा? प्रोफ़ाइलर या APM ट्रेस में लागत के आकार को नोटिस करके। - कौन-सा मेट्रिक इसे आते हुए दिखाता? ऑर्डर वॉल्यूम बढ़ने के साथ "निर्माण में बिताया समय" का "डोमेन लॉजिक में बिताया

समय" से बढ़ता अनुपात। - एक सक्षम इंजीनियर पहले क्या आजमाएगा, और क्यों अस्वीकार कर सकता है? अलग-अलग महँगी फ़ील्ड्स को कैश करना — एक उचित मध्यवर्ती क्रदम, लेकिन सामान्यीकृत नहीं होता।

The refactoring trigger: the insight that leads to Prototype is noticing the constructor is doing two jobs — assembling expensive shared state, and recording cheap instance-specific state — and splitting them along the `clone` boundary.

रीफ़ैक्टरिंग ट्रिगर: Prototype की ओर ले जाने वाली अंतर्दृष्टि यह नोटिस करना है कि कंस्ट्रक्टर दो काम कर रहा है — महँगी साझा स्थिति को इकट्ठा करना, और सस्ती इंस्टेंस-विशिष्ट स्थिति दर्ज करना — और उन्हें `clone` सीमा के साथ विभाजित करना।

✓ **Before you move on:** (1) What's the specific profiler signature that should make you suspect this problem? (2) Why is the caching-individual-fields approach a reasonable first step but not the structural fix?

✓ **आगे बढ़ने से पहले:** (1) कौन-सा खास प्रोफ़ाइलर हस्ताक्षर आपको इस समस्या पर शक करना चाहिए? (2) individual-fields कैशिंग तरीका एक उचित पहला क्रदम क्यों है लेकिन संरचनात्मक फ़िक्स क्यों नहीं?

Part 9 — Internal Working

This pattern *does* have a genuine internals story in PHP, so it's worth the space — but trimmed to exactly what explains the core gotcha.

इस पैटर्न की PHP में सचमुच एक असली आंतरिक कहानी है, इसलिए यह जगह के लायक है — लेकिन ठीक उतना ही जो मुख्य गड़बड़ी समझाए।

PHP objects are heap-allocated and accessed through reference-counted handles. `clone` allocates a *new* object on the heap and bitwise-copies the original's property values into it: scalar properties are copied by value automatically and are always safe; object-typed properties are copied as **shared handles** — unless `__clone()` says otherwise.

PHP ऑब्जेक्ट्स heap-allocated हैं और reference-counted हैंडल्स के ज़रिए एक्सेस होते हैं। `clone` heap पर एक नया ऑब्जेक्ट allocate करता है और मूल के प्रॉपर्टी मानों को bitwise-कॉपी करता है: स्केलर प्रॉपर्टीज़ स्वचालित रूप से वैल्यू से कॉपी होती हैं और हमेशा सुरक्षित हैं; ऑब्जेक्ट-टाइप प्रॉपर्टीज़ **साझा हैंडल्स** के तौर पर कॉपी होती हैं — जब तक `__clone()` कुछ और न कहे।

```
BEFORE clone: $original → [Customer #42] → address → [Address #7]
```

```
AFTER shallow clone (no __clone() override):
```

```
$original → [Customer #42] └─┐
                             └─┴─> BOTH point to [Address #7]
$copy      → [Customer #99] └─┘
```

```
Mutating $copy->address->city also changes what $original->address->city
reads – there is only ONE Address object, and both handles reference it.
```

`__clone()` runs **after** the shallow copy already exists, on the new object — it's a repair hook, not an interceptor. Its entire job is finding every object-typed property and explicitly re-cloning it, cascading into nested objects' own `__clone()` if the graph is more than one level deep.

`__clone()` **बाद में** चलता है, shallow कॉपी पहले से मौजूद होने के बाद, नए ऑब्जेक्ट पर — यह एक मरम्मत हुक है, कोई इंटरसेप्टर नहीं। इसका पूरा काम है हर ऑब्जेक्ट-टाइप प्रॉपर्टी ढूँढ़ना और खुलकर इसे फिर से क्लोन करना, अगर ग्राफ़ एक स्तर से ज़्यादा गहरा है तो nested ऑब्जेक्ट्स के अपने `__clone()` में कैस्केड होते हुए।

✓ **Before you move on:** (1) What PHP mechanism protects arrays from the shallow-copy problem, and why doesn't it protect object properties the same way? (2) When exactly does `__clone()` execute relative to the copy?

✓ **आगे बढ़ने से पहले:** (1) कौन-सा PHP तंत्र arrays को shallow-copy समस्या से बचाता है, और यह ऑब्जेक्ट प्रॉपर्टीज़ को उसी तरह क्यों नहीं बचाता? (2) `__clone()` कॉपी के सापेक्ष ठीक-ठीक कब निष्पादित होता है?

Part 10 — Components, UML & Language Mapping

Component	Responsibility
Prototype (interface, optional in PHP)	Declares the cloning contract
ConcretePrototype	Holds expensive-to-build shared state; implements <code>__clone()</code> to deep-copy mutable nested objects
Client	Requests a clone, supplies instance-specific overrides
PrototypeRegistry (common in production)	Maps a key to the correct pre-built prototype

कंपोनेंट्स: Prototype (इंटरफ़ेस, PHP में वैकल्पिक) — cloning कॉन्ट्रैक्ट डिक्लेयर करता है। ConcretePrototype — महुँगी-बनाने-वाली साझा स्थिति रखता है; परिवर्तनशील nested ऑब्जेक्ट्स को deep-copy करने के लिए `__clone()` इम्प्लीमेंट करता है। Client — एक क्लोन माँगता है, इंस्टेंस-विशिष्ट ओवरराइड्स देता है। PrototypeRegistry (प्रोडक्शन में आम) — एक की को सही पूर्व-निर्मित prototype से मैप करता है।

Class diagram:

```
«interface» Prototype
+-----+
| + clone(): static |
+-----+
      Δ
+-----+
|   InvoicePrototype   |
+-----+
| - taxProfile: TaxProfile | ← mutable, needs __clone() re-clone
| - pdfTemplate: string   | ← scalar, safe by default
| - orderId: int          | ← instance-specific
+-----+
| + __clone(): void       |
| + withOrder(...): static |
+-----+
```

A sequence diagram adds no real information beyond "clone, then overwrite fields" for this pattern — skipping it rather than including one for coverage's sake.

इस पैटर्न के लिए एक sequence डायग्राम "क्लोन करो, फिर फ़िल्ड्स ओवरराइट करो" से आगे कोई असली जानकारी नहीं जोड़ता — कवरेज के लिए एक शामिल करने के बजाय इसे छोड़ना।

Language mapping — the core mechanism, portable:

Language	Mechanism	Deep-copy responsibility
PHP	<code>clone</code> + <code>__clone()</code>	Manually re-clone object properties inside <code>__clone()</code>
Java	<code>Cloneable</code> + <code>clone()</code> override, or a copy constructor	<code>Object.clone()</code> is also shallow by default
Python	<code>copy.copy()</code> (shallow) vs. <code>copy.deepcopy()</code> (deep)	Python makes the shallow/deep distinction explicit in the API itself
Go	No built-in clone; manual struct copy	Pointer-typed fields are shared references after a naive struct copy
TypeScript/ Node	<code>Spread/ Object.assign()</code> for shallow; <code>structuredClone()</code> for deep	<code>structuredClone()</code> clones data but not class methods/prototypes

हिंदी सार (भाषा-मानचित्रण): PHP — `clone` + `__clone()`, ऑब्जेक्ट प्रॉपर्टीज़ को हाथ से फिर से क्लोन करना। Java — `Cloneable` + `clone()` ओवरराइट, `Object.clone()` भी डिफ़ॉल्ट रूप से shallow है। Python — `copy.copy()` (shallow) बनाम `copy.deepcopy()` (deep), API में ही स्पष्ट भेद। Go — कोई अंतर्निहित clone नहीं; मैनुअल struct कॉपी, pointer-टाइप फ़िल्ड्स साझा रेफ़रेंस रहते हैं। TypeScript/Node — shallow के लिए `Spread/ Object.assign()`; deep के लिए `structuredClone()`, जो डेटा क्लोन करता है, क्लास मेथड्स नहीं।

The gotcha is universal — every mainstream language's default/naive copy mechanism is shallow. This is worth saying explicitly if the interview isn't in PHP: the reasoning transfers completely, only the syntax changes.

गड़बड़ी सार्वभौमिक है — हर मुख्यधारा की भाषा का डिफ़ॉल्ट/सीधा-सादा कॉपी तंत्र shallow है। अगर इंटरव्यू PHP में नहीं है तो यह खुलकर कहने लायक है: तर्क पूरी तरह ट्रांसफ़र होता है, सिर्फ़ सिंटैक्स बदलता है।

✓ **Before you move on:** (1) Name the PHP mechanism and its closest Java equivalent. (2) Which language makes the shallow/deep distinction explicit in its API naming?

✓ **आगे बढ़ने से पहले:** (1) PHP तंत्र और इसका सबसे करीबी Java समकक्ष बताएँ। (2) कौन-सी भाषा अपने API नामकरण में shallow/deep भेद को खुलकर बताती है?

Part 11 — Implementation Overview (PHP/Laravel/Node)

All runnable code lives in `Prototype.php`. It progresses through three tiers: basic clone mechanics, the shallow-copy bug shown failing then fixed, and a production-shaped invoice-registry example.

सारा रनेबल कोड `Prototype.php` में है। यह तीन स्तरों से गुज़रता है: बुनियादी clone कार्यप्रणाली, विफल होते-फिर ठीक होते shallow-copy बग का प्रदर्शन, और एक प्रोडक्शन-आकार का invoice-registry उदाहरण।

Does Laravel use Prototype internally? — verified, not recalled. The commonly repeated claim is "Eloquent's `replicate()` is Prototype via `clone`." Having checked Laravel's actual source, **this is only half right, and the half that's wrong matters:** `replicate()` does **not** use PHP's `clone` / `__clone()` mechanism at all. It builds a new instance via `new static`, computes the attribute set to carry over by explicitly excluding the primary key and timestamp columns, sets those filtered attributes directly with `setRawAttributes()`, copies the loaded `$relations` array, and fires a `replicating` model event:

क्या Laravel आंतरिक रूप से Prototype इस्तेमाल करता है? — सत्यापित, याद से नहीं। आमतौर पर दोहराया गया दावा है "Eloquent का `replicate()`, `clone` के ज़रिए Prototype है।" Laravel के असली स्रोत की जाँच करने पर, **यह सिर्फ़ आधा सही है, और जो आधा ग़लत है वह मायने रखता है:** `replicate()` बिल्कुल भी PHP के `clone` / `__clone()` तंत्र का इस्तेमाल **नहीं** करता। यह `new static` के ज़रिए एक नया इंस्टेंस बनाता है, primary key और timestamp कॉलम्स को खुलकर बाहर रखकर ले जाने वाले attribute सेट की गणना करता है, `setRawAttributes()` से उन फ़िल्टर्ड attributes को सीधे सेट करता है, लोड किया गया `$relations` array कॉपी करता है, और एक `replicating` model इवेंट फ़ायर करता है:

```
public function replicate(?array $except = null)
{
    $defaults = array_values(array_filter([
        $this->getKeyName(), $this->getCreatedAtColumn(),
        $this->getUpdatedAtColumn(), ...$this->uniqueIds(), 'laravel_through_key',
    ]));
    $attributes = Arr::except($this->getAttributes(), $except ?
array_unique(array_merge($except, $defaults)) : $defaults);

    return tap(new static, function ($instance) use ($attributes) {
        $instance->setRawAttributes($attributes);
        $instance->setRelations($this->relations);
        $instance->fireModelEvent('replicating', false);
    });
}
```

Two things worth stating precisely in an interview: first, this is Prototype *in intent* but not Prototype *in mechanism* — no `clone` keyword involved. Second, `setRelations($this->relations)` copies the relations array by value, but any *related model objects already loaded inside it* are shared object references between the original and the replica — the exact shallow-copy gotcha from Part 9, present in real Laravel source.

इंटरव्यू में सटीक रूप से कहने लायक़ दो बातें: पहला, यह इरादे में Prototype है लेकिन तंत्र में Prototype नहीं — कोई `clone` कीवर्ड शामिल नहीं। दूसरा, `setRelations($this->relations)` relations array को वैल्यू से कॉपी करता है, लेकिन इसके

अंदर पहले से लोड की गई कोई भी संबंधित मॉडल ऑब्जेक्ट्स मूल और प्रतिकृति के बीच साझा ऑब्जेक्ट रेफरेंस हैं — Part 9 की ठीक वही shallow-copy गड़बड़ी, असली Laravel स्रोत में मौजूद।

Where Laravel genuinely doesn't use this pattern: the Service Container (Factory/DI-shaped) and Eloquent's query builder (Builder-shaped) dominate Laravel's actual creational-pattern usage.

Laravel सचमुच कहाँ इस पैटर्न का इस्तेमाल नहीं करता: Service Container (Factory/DI-आकार) और Eloquent का query builder (Builder-आकार) Laravel के असली क्रिएशनल-पैटर्न इस्तेमाल पर हावी हैं।

Node.js: no built-in `clone`; `{...obj} / Object.assign()` for shallow, `structuredClone()` (Node 17+) for deep — but `structuredClone()` clones data only, not class instances with methods.

Node.js: कोई अंतर्निहित `clone` नहीं; shallow के लिए `{...obj} / Object.assign()`, deep के लिए `structuredClone()` — लेकिन `structuredClone()` सिर्फ डेटा क्लोन करता है, मेथड्स वाले क्लास इंस्टेंसेज़ नहीं।

✓ **Before you move on:** (1) Does Laravel's `replicate()` use PHP's `clone` keyword? (2) What's the shallow-copy risk specific to `replicate()`'s handling of loaded relations?

✓ **आगे बढ़ने से पहले:** (1) क्या Laravel का `replicate()`, PHP के `clone` कीवर्ड का इस्तेमाल करता है? (2) `replicate()` के लोड किए गए relations के व्यवहार से जुड़ा खास shallow-copy जोखिम क्या है?

Part 12 — Where This Shows Up in Production

Scenario: flash-sale invoice generation. Branding, tax rules, and PDF templates are expensive and identical per (country, currency) pair; order data is cheap and always different. One `InvoicePrototype` per jurisdiction, registry-keyed, cloned per order.

परिदृश्य: फ़्लैश-सेल invoice जनरेशन। Branding, tax नियम, और PDF templates हर (country, currency) जोड़ी के लिए महँगे और एक जैसे हैं; ऑर्डर डेटा सस्ता और हमेशा अलग है। प्रति-क्षेत्राधिकार एक `InvoicePrototype`, registry-keyed, प्रति-ऑर्डर क्लोन किया गया।

Scenario: notification template fan-out. A billing event fans out to email/SMS/push, each needing a channel-specific but structurally similar object pre-loaded with branding, localization strings, and delivery config.

परिदृश्य: नोटिफ़िकेशन टेम्पलेट फ़ैन-आउट। एक बिलिंग इवेंट email/SMS/push तक फैलता है, हर एक को branding, स्थानीयकरण स्ट्रिंग्स, और डिलीवरी कॉन्फ़िग से पहले से लोडेड एक चैनल-विशिष्ट मगर संरचनात्मक रूप से समान ऑब्जेक्ट चाहिए।

Service	Plausible fit	Why
Invoice/Billing	Strong	Expensive shared config, high per-request volume
Notification	Strong	Same cost shape
Order (reorder feature)	Moderate	Maps directly to Laravel's <code>replicate()</code>
Payment	Anti-pattern	Cloning transaction state risks carrying over stale/incorrect state

Service	Plausible fit	Why
Auth/Session	Anti-pattern	Cloning session/token state risks leaking shared auth state

हिंदी सार: Invoice/Billing — मज़बूत फ़िट (महँगा साझा कॉन्फ़िग)। Notification — मज़बूत फ़िट (वही लागत आकार)। Order (reorder फ़ीचर) — मध्यम (Laravel के `replicate()` से सीधे मेल खाता है)। Payment — **एंटी-पैटर्न** (transaction स्थिति क्लोन करना बासी/गलत स्थिति ले जाने का जोखिम रखता है)। Auth/Session — **एंटी-पैटर्न** (session/token स्थिति क्लोन करना साझा auth स्थिति लीक करने का जोखिम रखता है)।

ADR — Architecture Decision Record (worked example):

ADR — आर्किटेक्चर डिज़ीज़न रिकॉर्ड (हल किया गया उदाहरण):

Title: Use a Prototype Registry for multi-region invoice generation **शीर्षक:** मल्टी-रीजन invoice जनरेशन के लिए एक Prototype Registry इस्तेमाल करें

Context: Invoice generation latency dominates order-processing p99 during high-volume windows. **संदर्भ:** उच्च-वॉल्यूम विंडोज़ के दौरान Invoice जनरेशन लेटेंसी order-processing p99 पर हावी है।

Decision: Introduce a `PrototypeRegistry` keyed by (country, currency). **फ़ैसला:** (country, currency) से keyed एक `PrototypeRegistry` पेश करें।

Alternatives considered: (1) Cache individual fields — rejected as treating the symptom. (2) Precompute fully-rendered invoices — rejected because order-specific data is unique. **विचार किए गए विकल्प:** (1) individual फ़िल्ड्स कैश करना — लक्षण का इलाज करने के तौर पर अस्वीकार। (2) पूरी तरह-रेंडर्ड invoices को precompute करना — अस्वीकार क्योंकि ऑर्डर-विशिष्ट डेटा अद्वितीय है।

Consequences: Construction cost drops from $O(n)$ to $O(1)$ + cheap clones. New operational responsibility: registry staleness must be actively invalidated. **नतीजे:** निर्माण लागत $O(n)$ से $O(1)$ + सस्ती क्लोन्स तक गिरती है। नई परिचालन ज़िम्मेदारी: registry बासीपन को सक्रिय रूप से invalidate किया जाना चाहिए।

Trade-offs: Correctness now depends on `__clone()` being exhaustively correct. **ट्रेड-ऑफ़्स:** सटीकता अब इस पर निर्भर करती है कि `__clone()` पूरी तरह से सही हो।

✓ **Before you move on:** (1) Name one service where Prototype is a genuine anti-pattern, and why. (2) In the ADR, what alternative was rejected, and on what basis?

✓ **आगे बढ़ने से पहले:** (1) एक सेवा बताएँ जहाँ Prototype एक असली एंटी-पैटर्न है, और क्यों। (2) ADR में, कौन-सा विकल्प अस्वीकार किया गया, और किस आधार पर?

Part 13 — Field Notes (Simulated Production Experience)

This is a rehearsal scaffold, not a script. Personalize it with details from your own projects before using it as an interview answer.

यह एक रिहर्सल ढाँचा है, कोई स्क्रिप्ट नहीं। इसे इंटरव्यू जवाब के तौर पर इस्तेमाल करने से पहले अपने प्रोजेक्ट्स के विवरणों से निजीकृत करें।

"On a multi-region invoicing service, invoice generation was originally a per-order constructor call pulling branding, tax rules, and a PDF template inline. It worked at normal volume. During a regional flash-sale event, order volume spiked, and invoice generation became the dominant contributor to p99 order-processing latency. Profiling showed the same branding/tax/template fetches repeating identically across thousands of consecutive orders.

The fix was a `PrototypeRegistry` keyed by (country, currency). The registry subscribed to a `TaxRulesUpdated` domain event to invalidate and rebuild the relevant entry.

The part that actually required care wasn't the cloning mechanism — it was correctly identifying every mutable nested object that needed an explicit deep-copy inside `__clone()`. We initially missed one, and it surfaced as an intermittent bug where one customer's invoice adjustment appeared to leak into another's under concurrent load."

"एक मल्टी-रीजन invoicing सेवा में, invoice जनरेशन मूल रूप से एक प्रति-ऑर्डर कंस्ट्रक्टर कॉल थी जो branding, tax नियम, और एक PDF template इनलाइन खींचती थी। सामान्य वॉल्यूम पर यह ठीक चलता था। एक क्षेत्रीय फ्लैश-सेल इवेंट के दौरान, ऑर्डर वॉल्यूम बढ़ा, और invoice जनरेशन p99 order-processing लेटेंसी का मुख्य योगदानकर्ता बन गया। प्रोफाइलिंग ने दिखाया कि वही branding/tax/template फ़ेच हज़ारों लगातार ऑर्डर्स में एक जैसे दोहरा रहे थे।

फ़िक्स था (country, currency) से keyed एक `PrototypeRegistry`। Registry ने संबंधित एंट्री को invalidate और फिर से बनाने के लिए एक `TaxRulesUpdated` डोमेन इवेंट सब्सक्राइब किया।

जिस हिस्से को असल में सावधानी चाहिए थी वह cloning तंत्र नहीं था — यह हर उस परिवर्तनशील nested ऑब्जेक्ट को सही ढंग से पहचानना था जिसे `__clone()` के अंदर एक खुली deep-copy चाहिए थी। हम शुरू में एक चूक गए, और यह एक रुक-रुक कर आने वाली बग के तौर पर सामने आया जहाँ एक ग्राहक का invoice समायोजन समवर्ती लोड के तहत दूसरे में लीक होता दिखा।"

✓ **Before you move on:** (1) What specifically caused the production incident in this account — the pattern choice, or the implementation? (2) What single artifact would have caught it before shipping?

✓ **आगे बढ़ने से पहले:** (1) इस वृत्तांत में असल में प्रोडक्शन घटना का कारण क्या था — पैटर्न चुनाव, या इम्प्लीमेंटेशन? (2) कौन-सा एक आर्टिफैक्ट इसे शिप करने से पहले पकड़ लेता?

Part 14 — Analogies & Architecture Fit

Analogies: - **Passport office** — master template (logo, watermark, QR layout) stays constant; each passport is a "clone" with name/DOB/number changed. - **Rubber stamp** — carving = expensive one-time construction; stamping = cheap repeated cloning. - **Photocopier with a master document** — you copy the master and annotate the copy. - **Cell division** — a cell copies existing DNA rather than building it from raw materials.

उपमाएँ: - **पासपोर्ट कार्यालय** — मास्टर टेम्पलेट (लोगो, वॉटरमार्क, QR लेआउट) स्थिर रहता है; हर पासपोर्ट नाम/जन्म-तिथि/नंबर बदले हुए एक "क्लोन" है। - **रबर स्टैम्प** — तराशना = महँगा एक-बार निर्माण; स्टैम्प लगाना = सस्ती दोहराई गई cloning। - **मास्टर**

दस्तावेज़ वाली फ़ोटोकॉपीयर — आप मास्टर की कॉपी करते हैं और कॉपी पर टिप्पणी करते हैं। - **कोशिका विभाजन** — एक कोशिका कच्चे पदार्थों से बनाने के बजाय मौजूदा DNA की कॉपी करती है।

Architecture fit: - **Clean/Hexagonal/Onion:** belongs in the domain/application layer — cloning is a domain-level decision about object identity and cost. - **DDD:** maps cleanly onto Value Objects that are expensive to construct. - **Event-driven architecture:** a registry's invalidation trigger is a natural event-consumer role. - **CQRS:** no meaningful connection — stated plainly rather than forced. - **Cloud-native/Kubernetes:** the *principle* underlies manifest templating broadly (Helm charts, Terraform modules), but this is a principle-level analogy, not literal object cloning.

आर्किटेक्चर फ़िट: - **Clean/Hexagonal/Onion:** डोमेन/एप्लिकेशन लेयर में आता है — cloning ऑब्जेक्ट पहचान और लागत के बारे में एक डोमेन-स्तर का फैसला है। - **DDD:** उन Value Objects पर साफ़ तौर पर मैप होता है जो बनाने में महँगे हैं। - **इवेंट-ड्रिवन आर्किटेक्चर:** एक registry का invalidation ट्रिगर एक स्वाभाविक event-consumer भूमिका है। - **CQRS:** कोई सार्थक संबंध नहीं — खुलकर कहा गया, थोपा नहीं गया। - **Cloud-native/Kubernetes:** सिद्धांत व्यापक रूप से manifest templating (Helm charts, Terraform modules) के आधार में है, लेकिन यह एक सिद्धांत-स्तर की उपमा है, शाब्दिक ऑब्जेक्ट cloning नहीं।

✓ **Before you move on:** (1) Which analogy best captures "pay once, reuse many," and why? (2) Which architecture style has no meaningful connection to this pattern?

✓ **आगे बढ़ने से पहले:** (1) कौन-सी उपमा "एक बार चुकाओ, कई बार इस्तेमाल करो" को सबसे अच्छी तरह पकड़ती है, और क्यों? (2) किस आर्किटेक्चर शैली का इस पैटर्न से कोई सार्थक संबंध नहीं है?

Part 15 — SOLID, Performance & Concurrency

SOLID: SRP and OCP get a strong, positive connection — construction logic separates cleanly from "produce a new instance," and new variants are added via a new registry entry, not by modifying existing classes. LSP requires discipline if `ConcretePrototype` subclasses exist. ISP and DIP have no meaningful connection worth forcing here.

SOLID: SRP और OCP का एक मज़बूत, सकारात्मक संबंध है — निर्माण लॉजिक "एक नया इंस्टेंस बनाओ" से साफ़ तौर पर अलग होता है, और नए वेरिएंट्स एक नई registry एंट्री के ज़रिए जोड़े जाते हैं, मौजूदा क्लासेज़ बदलकर नहीं। LSP को अनुशासन चाहिए अगर `ConcretePrototype` सबक्लासेज़ मौजूद हैं। ISP और DIP का यहाँ थोपने लायक कोई सार्थक संबंध नहीं।

Performance: without Prototype, N instances cost $O(N \times \text{construction_cost})$; with it, $O(1 \times \text{construction_cost}) + O(N \times \text{clone_cost})$. This is a reasoned estimate, not a benchmark; validate with real profiling before quoting a specific number.

परफॉर्मेंस: Prototype के बिना, N इंस्टेंसेज़ की लागत $O(N \times \text{construction_cost})$ है; इसके साथ, $O(1 \times \text{construction_cost}) + O(N \times \text{clone_cost})$ । यह एक तर्कसंगत अनुमान है, कोई बेंचमार्क नहीं; कोई खास संख्या बताने से पहले असली प्रोफाइलिंग से सत्यापित करें।

Concurrency: under standard PHP-FPM, a registry read is safe without additional locking. This changes under long-running processes — Swoole, RoadRunner, persistent queue workers — where a registry instance persists across many requests. If `__clone()` is incomplete, two

logically-independent operations can end up mutating the same underlying nested object — a genuine race condition. In Node.js, the single-threaded event loop doesn't remove the equivalent risk.

Concurrency: मानक PHP-FPM के तहत, एक registry रीड बिना अतिरिक्त लॉकिंग के सुरक्षित है। यह लंबे समय तक चलने वाली प्रोसेसेज़ — Swoole, RoadRunner, स्थायी queue वर्कर्स — के तहत बदल जाता है, जहाँ एक registry इंस्टेंस कई रिक्वेस्ट्स में बना रहता है। अगर `__clone()` अधूरा है, तो दो तार्किक रूप से स्वतंत्र ऑपरेशन्स एक ही अंतर्निहित nested ऑब्जेक्ट को बदल सकते हैं — एक असली रेस कंडीशन। Node.js में, सिंगल-थ्रेडेड इवेंट लूप समकक्ष जोखिम को नहीं हटाता।

✓ **Before you move on:** (1) Which two SOLID principles have no meaningful connection to this pattern? (2) Under which specific PHP deployment models does an incomplete `__clone()` become a concurrency bug?

✓ **आगे बढ़ने से पहले:** (1) किन दो SOLID सिद्धांतों का इस पैटर्न से कोई सार्थक संबंध नहीं है? (2) किन खास PHP डिप्लॉयमेंट मॉडल्स के तहत एक अधूरा `__clone()` एक concurrency बग बन जाता है?

Part 16 — Advantages, Disadvantages & Trade-offs

Dimension	Advantage	Disadvantage / trade-off
Performance	Converts $O(N)$ construction into $O(1)$ + cheap copies	Pure overhead for cheap-to-construct objects
Scalability	Improves throughput under load	A shared registry can itself become a bottleneck if <code>__clone()</code> is incomplete
Maintainability	Centralizes construction logic	Every new mutable nested property is a new place a deep-copy can be forgotten
Readability	Well-understood, named pattern once recognized	Slightly less direct than a plain <code>new</code>
Security	Neutral-to-positive for config/template-shaped data	Actively risky if misapplied to session/auth/transaction-shaped data
Testing	Construction logic is testable once, in isolation	Adds a real new testing obligation — clone-independence tests
Observability	A registry is a natural place to add metrics	Registry staleness is a new operational failure mode

हिंदी सार: परफॉर्मेंस — $O(N)$ निर्माण को $O(1)$ + सस्ती कॉपियों में बदलता है, सस्ते ऑब्जेक्ट्स के लिए शुद्ध ओवरहेड। स्केलेबिलिटी — लोड के तहत थ्रूपुट सुधारता है, लेकिन एक साझा registry खुद एक बाधा बन सकती है। मंटेनेबिलिटी — निर्माण लॉजिक केंद्रीकृत करता है, लेकिन हर नई परिवर्तनशील nested प्रॉपर्टी एक नई जगह है जहाँ deep-copy भूली जा सकती है। पठनीयता — एक बार पहचाने जाने पर सादा, अच्छी तरह समझा गया पैटर्न। सुरक्षा — config/template-आकार के डेटा के लिए तटस्थ-से-सकारात्मक, लेकिन session/auth/transaction-आकार के डेटा पर ग़लत लगाने पर सक्रिय रूप से जोखिम भरा। टेस्टिंग — निर्माण लॉजिक एक

बार, अलग-थलग टेस्ट करने-योग्य, लेकिन एक असली नई टेस्टिंग जिम्मेदारी जोड़ता है। ऑब्जेक्टिविलिटी — मैट्रिक्स जोड़ने के लिए एक स्वाभाविक जगह, लेकिन registry बासीपन एक नया परिचालन विफलता ढंग है।

✓ **Before you move on:** (1) Name one dimension where this pattern is a clear net positive with no real downside. (2) Name one dimension where the trade-off genuinely could go either way.

✓ **आगे बढ़ने से पहले:** (1) एक आयाम बताएँ जहाँ यह पैटर्न बिना किसी असली नुकसान के साफ़ तौर पर फ़ायदेमंद है। (2) एक आयाम बताएँ जहाँ ट्रेड-ऑफ़ सचमुच किसी भी तरफ़ जा सकता है।

Part 17 — Pattern Comparisons

	Prototype	Factory	Builder	Singleton
Mechanism	<code>clone</code> an existing instance	<code>new</code> via creation logic/branching	Step-by-step assembly, fluent API	Exactly one instance, ever
Best for	Expensive, structurally-similar instances	Deciding <i>which</i> concrete type to build	Objects with many optional/combinable parameters	A single shared instance globally
Relationship to <code>clone</code>	Central mechanism	Unrelated	Unrelated	Actively blocks it
Cooperates with Prototype?	—	Often	Rarely relevant together	No — opposite instinct

हिंदी सार (तुलना टेबल): Prototype में एक मौजूदा इंस्टेंस को `clone` करना तंत्र है, महुँगी, संरचनात्मक रूप से समान इंस्टेंसेज़ के लिए सबसे अच्छा। Factory में creation लॉजिक/शाखाबद्धता के ज़रिए `new` है, यह तय करने के लिए सबसे अच्छा कि कौन-सी कॉन्क्रीट टाइप बनानी है। Builder में चरण-दर-चरण असेंबली है, कई वैकल्पिक पैरामीटर्स वाले ऑब्जेक्ट्स के लिए सबसे अच्छा। Singleton में हमेशा ठीक एक इंस्टेंस है, और यह `clone` को सक्रिय रूप से ब्लॉक करता है।

Memento, briefly: commonly confused because both "involve copying state," but the intent differs — Memento captures one object's internal state *for later restoration of that same object* (undo/redo); Prototype captures state *to seed brand-new, independent objects going forward*.

Memento, संक्षेप में: आमतौर पर गड़मड़ होता है क्योंकि दोनों में "स्थिति कॉपी करना" शामिल है, लेकिन इरादा अलग है — Memento एक ऑब्जेक्ट की आंतरिक स्थिति को उसी ऑब्जेक्ट को बाद में बहाल करने के लिए पकड़ता है (undo/redo); Prototype स्थिति को आगे बिल्कुल नए, स्वतंत्र ऑब्जेक्ट्स बीजने के लिए पकड़ता है।

Decision table:

Situation	Reach for
Construction is expensive; instances are near-identical	Prototype
Concrete type decided at runtime via branching logic	Factory

Situation	Reach for
Many optional, combinable construction parameters	Builder
Exactly one instance must ever exist	Singleton
Snapshot and later restore one specific object's past state	Memento
A Factory needs to build the initial object that Prototype then clones	Both, together

निर्णय टेबल — हिंदी सार: निर्माण महँगा है; इंस्टेंसेज़ लगभग एक जैसी हैं → Prototype। कॉन्क्रीट टाइप रनटाइम पर शाखाबद्ध लॉजिक से तय होती है → Factory। कई वैकल्पिक, संयोजनीय निर्माण पैरामीटर्स → Builder। हमेशा ठीक एक इंस्टेंस मौजूद होना चाहिए → Singleton। एक खास ऑब्जेक्ट की पिछली स्थिति स्नैपशॉट और बाद में बहाल करना → Memento। एक Factory को शुरुआती ऑब्जेक्ट बनाना है जिसे Prototype फिर क्लोन करता है → दोनों, एक साथ।

✓ **Before you move on:** (1) Why does Singleton typically make `__clone()` private? (2) What's the one-sentence distinction between Prototype and Memento?

✓ **आगे बढ़ने से पहले:** (1) Singleton आमतौर पर `__clone()` को प्राइवेट क्यों बनाता है? (2) Prototype और Memento के बीच एक-वाक्य का भेद क्या है?

Part 18 — Production Bugs, AI-Generated Code Review & Testing

The flagship bug — shallow-copy leak. A mutable nested object property wasn't deep-copied in `__clone()`. Symptom: mutating what looks like an independent clone silently changes data on a different object. Debug by reproducing with an identity check (`===` , not `==`).

मुख्य बग — shallow-copy लीक। एक परिवर्तनशील nested ऑब्जेक्ट प्रॉपर्टी `__clone()` में deep-copy नहीं की गई। लक्षण: एक स्वतंत्र क्लोन जैसी दिखने वाली चीज़ को बदलना चुपचाप एक अलग ऑब्जेक्ट पर डेटा बदल देता है। एक पहचान जाँच (`===` , `==` नहीं) से दोहराकर डीबग करें।

Stale registry entries. New orders keep using outdated tax rates or branding after a legitimate business change, with no crash. Fix: event-driven invalidation, not a shorter TTL.

बासी registry एंट्रीज़। एक वैध व्यावसायिक बदलाव के बाद नए ऑर्डर्स पुराने tax दरों या branding का इस्तेमाल करते रहते हैं, बिना किसी क्रैश के। फ़िक्स: event-driven invalidation, कोई छोटा TTL नहीं।

How AI coding assistants typically get this pattern wrong: - **Most common failure:** AI-generated `clone / __clone()` implementations frequently handle the *first* level of an object graph correctly but don't cascade into a second level. - **Second most common failure:** AI-suggested "Prototype" implementations for scenarios that are actually Factory problems. -

What a reviewer should check before merging: (1) does `__clone()` cascade to every level; (2) is the pattern actually justified by "expensive construction + similar instances"; (3) does the generated code clone anything from the anti-trigger list.

AI कोडिंग असिस्टेंट्स आमतौर पर इस पैटर्न को कैसे ग़लत करते हैं: - सबसे आम विफलता: AI-जनित `clone / __clone()` इम्प्लीमेंटेशन्स अक्सर एक ऑब्जेक्ट ग्राफ़ के पहले स्तर को सही ढंग से संभालती हैं लेकिन दूसरे स्तर में कैस्केड नहीं होतीं। - दूसरी सबसे आम विफलता: AI-सुझाए "Prototype" इम्प्लीमेंटेशन्स उन परिदृश्यों के लिए जो असल में Factory समस्याएँ हैं। - **मर्ज करने से पहले एक रिव्यूअर को क्या जाँचना चाहिए:** (1) क्या `__clone()` हर स्तर तक कैस्केड होता है; (2) क्या पैटर्न असल में "महंगा निर्माण + समान इंस्टेंसेज़" से जायज़ है; (3) क्या जनित कोड एंटी-ट्रिगर सूची से कुछ भी क्लोन करता है।

Testing strategy — the clone-independence test is the one category that matters most for this pattern:

टेस्टिंग रणनीति — clone-independence टेस्ट वह एक श्रेणी है जो इस पैटर्न के लिए सबसे ज़्यादा मायने रखती है:

```
public function test_cloning_produces_a_fully_independent_object(): void
{
    $prototype = InvoicePrototype::fromConfig($fakeConfig);
    $cloneA = clone $prototype;
    $cloneB = clone $prototype;

    $cloneA->taxProfile->rate = 0.99;

    $this->assertNotSame($cloneA->taxProfile, $cloneB->taxProfile); // identity, not
value
    $this->assertNotSame($cloneA->taxProfile, $prototype->taxProfile);
    $this->assertNotEquals(0.99, $cloneB->taxProfile->rate);
}
```

The critical detail: `assertNotSame` (identity) on every nested object property, not `assertNotEquals` on values alone — a value check can pass right up until the moment one clone is mutated.

महत्वपूर्ण विवरण: हर nested ऑब्जेक्ट प्रॉपर्टी पर `assertNotSame` (पहचान), सिर्फ़ मानों पर `assertNotEquals` नहीं — एक वैल्यू जाँच तब तक पास हो सकती है जब तक एक क्लोन को बदला न जाए।

Code review checklist: every object-typed property has a corresponding `__clone()` line; a clone-independence test exists for every mutable nested property; nothing from the anti-trigger list is being cloned; any registry entry sourced from mutable external data has a defined invalidation path.

कोड रिव्यू चेकलिस्ट: हर ऑब्जेक्ट-टाइप प्रॉपर्टी की एक मेल खाती `__clone()` पंक्ति है; हर परिवर्तनशील nested प्रॉपर्टी के लिए एक clone-independence टेस्ट मौजूद है; एंटी-ट्रिगर सूची से कुछ भी क्लोन नहीं हो रहा; परिवर्तनशील बाहरी डेटा से बनी किसी भी registry एंट्री का एक परिभाषित invalidation पथ है।

✓ **Before you move on:** (1) What's the single most common way AI tools get this pattern wrong? (2) Why must the clone-independence test use `assertNotSame`, not `assertEquals`?

✓ **आगे बढ़ने से पहले:** (1) AI टूल्स इस पैटर्न को सबसे आम तरीक़े से कैसे ग़लत करते हैं? (2) clone-independence टेस्ट को `assertEquals` नहीं, `assertNotSame` का इस्तेमाल क्यों करना चाहिए?

Part 19 — Refactoring Journey

Full code for every stage lives in `Prototype.php`; this narrates the reasoning connecting each one.

हर चरण का पूरा कोड `Prototype.php` में है; यह हर एक को जोड़ने वाला तर्क बताता है।

Stage 1 — Terrible (*where most engineers start, no shame in it*): everything rebuilt inline, every call, no separation of concerns. Works, doesn't scale.

चरण 1 — भयानक (जहाँ से ज़्यादातर इंजीनियर शुरू करते हैं, इसमें कोई शर्म नहीं): हर कॉल में सब कुछ इनलाइन फिर से बनाया गया, कोई चिंताओं का पृथक्करण नहीं। काम करता है, स्केल नहीं करता।

Stage 2 — Bad, but a realistic first instinct (*often written by a mid-level engineer under time pressure*): hand-rolled static caching of individual constructor arguments. Papers over the symptom without addressing the structural cause.

चरण 2 — बुरा, मगर एक यथार्थवादी पहला अंतर्ज्ञान (अक्सर समय के दबाव में एक मिड-लेवल इंजीनियर द्वारा लिखा गया): individual कंस्ट्रक्टर आर्ग्युमेंट्स की हाथ से बनाई स्टैटिक कैशिंग। संरचनात्मक कारण को संबोधित किए बिना लक्षण को ढाँकना।

Stage 3 — Average, and the most dangerous stage in the whole journey (*a senior engineer moving fast, or code that later drifts*): correctly splits expensive-shared from cheap-instance-specific construction, but implements only a shallow clone with no `__clone()` override. Passes casual testing, looks finished, silently carries the shallow-copy bug.

चरण 3 — औसत, और पूरी यात्रा का सबसे खतरनाक चरण (एक सीनियर इंजीनियर तेज़ी से आगे बढ़ रहा है, या कोड जो बाद में भटक जाता है): मँहगे-साझा को सस्ते-इंस्टेंस-विशिष्ट निर्माण से सही ढंग से अलग करता है, लेकिन बिना `__clone()` ओवरराइड के सिर्फ़ एक shallow clone इम्प्लीमेंट करता है। सामान्य टेस्टिंग पास करता है, पूरा दिखता है, चुपचाप shallow-copy बग ले जाता है।

Stage 4 — Pattern correctly applied (*what a rigorous senior/staff engineer ships*): adds a correctly-cascading `__clone()` plus a clone-independence test proving it. Functionally complete and correct.

चरण 4 — पैटर्न सही ढंग से लगाया गया (जो एक सख्त सीनियर/स्टाफ़ इंजीनियर शिप करता है): एक सही ढंग से कैस्केड होने वाला `__clone()` जोड़ता है, साथ ही इसे साबित करने वाला clone-independence टेस्ट। कार्यात्मक रूप से पूरा और सही।

Stage 5 — Production-ready (*staff-level judgment about the surrounding system, not just the class*): wraps the correct implementation in a keyed registry with event-driven invalidation, instrumented with entry-age and clone-count metrics.

चरण 5 — प्रोडक्शन-रेडी (आस-पास के सिस्टम के बारे में स्टाफ़-स्तर का निर्णय, सिर्फ़ क्लास के बारे में नहीं): सही इम्प्लीमेंटेशन को event-driven invalidation वाली एक keyed registry में लपेटता है, entry-age और clone-count मेट्रिक्स से इंस्ट्रुमेंटेड।

✓ **Before you move on:** (1) Which stage is the most dangerous to leave in production? (2) What distinguishes Stage 4 from Stage 5 — is it a code difference or a systems difference?

✓ **आगे बढ़ने से पहले:** (1) कौन-सा चरण प्रोडक्शन में छोड़ने के लिए सबसे खतरनाक है? (2) चरण 4 और चरण 5 में क्या अंतर है — क्या यह एक कोड अंतर है या एक सिस्टम अंतर?

Part 20 — Practices, Mistakes & Traps

Junior mistakes: believing `clone` re-runs `__construct()`; reaching for Prototype on trivially cheap objects; not knowing `__clone()` exists.

शुरुआती गलतियाँ: यह मानना कि `clone`, `__construct()` फिर से चलाता है; तुच्छ रूप से सस्ते ऑब्जेक्ट्स पर Prototype की ओर पहुँचना; यह न जानना कि `__clone()` मौजूद है।

Mid-level mistakes: implementing `__clone()` for the top level but forgetting it must cascade into nested objects' own `__clone()`; assuming "I overrode `__clone()`" automatically means "it's fully deep now"; building a registry with no invalidation strategy.

मिड-लेवल गलतियाँ: ऊपरी स्तर के लिए `__clone()` इम्प्लीमेंट करना लेकिन यह भूलना कि इसे nested ऑब्जेक्ट्स के अपने `__clone()` में कैसेकेड होना चाहिए; यह मान लेना कि "मैंने `__clone()` ओवरराइड किया" स्वचालित रूप से मतलब "यह अब पूरी तरह deep है"; बिना किसी invalidation रणनीति के एक registry बनाना।

Senior mistakes — subtler, and the ones that actually distinguish levels in an interview: applying Prototype to session/auth/transaction-shaped objects because the *mechanism* technically fits; missing the concurrency escalation specific to long-running processes; stating a performance claim as benchmarked fact without having actually measured it.

सीनियर गलतियाँ — ज़्यादा सूक्ष्म, और वे जो इंटरव्यू में असल में स्तरों को अलग करती हैं: session/auth/transaction-आकार के ऑब्जेक्ट्स पर Prototype लगाना क्योंकि तंत्र तकनीकी रूप से फिट बैठता है; लंबे समय तक चलने वाली प्रोसेसेज़ के लिए खास concurrency वृद्धि को चूक जाना; बिना असल में मापे एक परफॉर्मंस दावे को बेंचमार्क तथ्य के तौर पर बताना।

Interview traps — the specific follow-ups that catch memorized-but-shallow

understanding: - "So cloning is always faster than `new`, right?" — agreeing unconditionally is the trap. - "Fix this shallow-copy bug" (live) — the trap is fixing only the first-level nested property. - "Is PHP thread-safe here?" — a blanket "PHP is single-threaded" answer is the trap. - "Would you clone this session object?" — testing judgment, not mechanism. - "So Prototype replaces Factory?" — framing them as competitors instead of noting they cooperate.

इंटरव्यू जाल — वे खास फ़ॉलो-अप्स जो रटी-मगर-सतही समझ पकड़ते हैं: - "तो cloning हमेशा `new` से तेज़ है, है ना?" — बिना शर्त सहमत होना जाल है। - "इस *shallow-copy* बग को ठीक करो" (लाइव) — जाल सिर्फ़ पहले-स्तर की nested प्रॉपर्टी ठीक करना है। - "क्या PHP यहाँ थ्रेड-सेफ़ है?" — एक सामान्य "PHP सिंगल-थ्रेडेड है" जवाब जाल है। - "क्या आप इस *session* ऑब्जेक्ट को क्लोन करेंगे?" — निर्णय जाँचना, तंत्र नहीं। - "तो *Prototype*, *Factory* की जगह लेता है?" — उन्हें प्रतिस्पर्धियों के तौर पर फ्रेम करना, यह बताने के बजाय कि वे सहयोग करते हैं।

✓ **Before you move on:** (1) Which junior mistake is purely a language-mechanics gap, not a design judgment gap? (2) Pick one interview trap above and state the correct answer in one sentence.

✓ **आगे बढ़ने से पहले:** (1) कौन-सी शुरुआती गलती सिर्फ़ एक भाषा-कार्यप्रणाली गैप है, कोई डिज़ाइन निर्णय गैप नहीं? (2) ऊपर से एक इंटरव्यू जाल चुनें और एक वाक्य में सही जवाब बताएँ।

Part 21 — Interview Question Bank & Coding Problems

22 questions total across five levels — intentionally below a flat quota, because this pattern's real interview footprint (Part 2) doesn't justify padding it to match a Very High priority pattern's bank.

पाँच स्तरों में कुल 22 सवाल — जानबूझकर एक सपाट कोटे से कम, क्योंकि इस पैटर्न का असली इंटरव्यू फुटप्रिंट (Part 2) इसे एक Very High प्राथमिकता वाले पैटर्न के बैंक जितना बड़ा बनाने को सही नहीं ठहराता।

Beginner (5)

B1. What is the Prototype pattern? *Wrong:* "Copying an object." *Good:* "A creational pattern that clones an existing instance instead of building from scratch." *Excellent:* Good + "used specifically when construction is expensive or many similar instances are needed quickly." *Follow-up:* Name the other four creational GoF patterns.

हिंदी: Prototype पैटर्न क्या है? — ग़लत: "एक ऑब्जेक्ट कॉपी करना।" — अच्छा: "एक क्रिएशनल पैटर्न जो शुरू से बनाने के बजाय एक मौजूदा इंस्टेंस को क्लोन करता है।" — उत्कृष्ट: अच्छा + "खास तौर पर तब इस्तेमाल होता है जब निर्माण महँगा हो या जल्दी कई समान इंस्टेंसेज़ चाहिए हों।" — फ़ॉलो-अप: बाक़ी चार क्रिएशनल GoF पैटर्न्स के नाम बताइए।

B2. Does `clone` call the constructor in PHP? *Wrong:* Yes. *Good:* "No — it bitwise-copies properties and calls `__clone()` if defined." *Excellent:* Good + a concrete example. *Follow-up:* What does `__clone()` actually do, and when does it run?

हिंदी: क्या PHP में `clone`, कंस्ट्रक्टर कॉल करता है? — ग़लत: हाँ। — अच्छा: "नहीं — यह प्रॉपर्टीज़ को bitwise-कॉपी करता है और अगर डिफ़ाइन हो तो `__clone()` कॉल करता है।" — उत्कृष्ट: अच्छा + एक ठोस उदाहरण। — फ़ॉलो-अप: `__clone()` असल में क्या करता है, और यह कब चलता है?

B3. Is PHP's `clone` shallow or deep by default? *Wrong:* Deep. *Good:* "Shallow — scalars by value, objects by shared reference." *Excellent:* Good + a concrete bug example and the fix. *Follow-up:* Show the fix live.

हिंदी: क्या PHP का `clone` डिफ़ॉल्ट रूप से shallow है या deep? — ग़लत: Deep। — अच्छा: "Shallow — स्केलर्स वैल्यू से, ऑब्जेक्ट्स साझा रेफ़रेंस से।" — उत्कृष्ट: अच्छा + एक ठोस बग उदाहरण और फ़िक्स। — फ़ॉलो-अप: फ़िक्स लाइव दिखाइए।

B4. When would you NOT use Prototype? *Wrong:* No real answer. *Good:* "When the object is cheap to construct." *Excellent:* Good + the session/transaction-state anti-trigger. *Follow-up:* What would you use instead if concrete types genuinely vary?

हिंदी: आप Prototype कब इस्तेमाल नहीं करेंगे? — ग़लत: कोई असली जवाब नहीं। — अच्छा: "जब ऑब्जेक्ट बनाना सस्ता हो।" — उत्कृष्ट: अच्छा + session/transaction-स्थिति एंटी-ट्रिगर। — फ़ॉलो-अप: अगर कॉन्क्रीट टाइप्स सचमुच बदलती हैं तो आप इसके बजाय क्या इस्तेमाल करेंगे?

B5. Name one production use case. *Wrong:* A Shape/Animal toy example. *Good:* Invoice generation with shared branding/tax config. *Excellent:* Good + explains specifically what's expensive about the shared part. *Follow-up:* What goes wrong if the cloning is implemented incorrectly there?

हिंदी: एक प्रोडक्शन इस्तेमाल का मामला बताइए। — ग़लत: एक Shape/Animal खिलौना उदाहरण। — अच्छा: साझा branding/tax कॉन्फ़िग वाला invoice जनरेशन। — उत्कृष्ट: अच्छा + खास तौर पर बताता है कि साझा हिस्से के बारे में क्या महंगा है। — फ़ॉलो-अप: अगर वहाँ cloning ग़लत तरीके से इम्प्लीमेंट हो तो क्या गड़बड़ होती है?

Intermediate (5)

11. Walk me through fixing a shallow-copy bug. *Wrong:* Vague "just override clone." *Good:* Identify the shared nested object, override `__clone()`, explicitly re-clone it. *Excellent:* Good + writes an identity-check test. *Follow-up:* What if that nested object has its own nested objects?

हिंदी: मुझे एक shallow-copy बग ठीक करने से गुज़ारें। — ग़लत: अस्पष्ट "बस clone ओवरराइड करो।" — अच्छा: साझा nested ऑब्जेक्ट पहचानें, `__clone()` ओवरराइड करें, खुलकर इसे फिर से क्लोन करें। — उत्कृष्ट: अच्छा + एक पहचान-जाँच टेस्ट लिखता है। — फ़ॉलो-अप: अगर उस nested ऑब्जेक्ट के अपने nested ऑब्जेक्ट्स हों तो?

12. How does Prototype relate to Factory? *Wrong:* "They're alternatives, pick one." *Good:* Correctly states they solve different problems. *Excellent:* Good + names that a Factory often builds the initial prototype. *Follow-up:* Design a system using both together.

हिंदी: Prototype, Factory से कैसे संबंधित है? — ग़लत: "वे विकल्प हैं, एक चुनो।" — अच्छा: सही ढंग से कहता है कि वे अलग समस्याएँ हल करते हैं। — उत्कृष्ट: अच्छा + बताता है कि एक Factory अक्सर शुरुआती prototype बनाती है। — फ़ॉलो-अप: दोनों को एक साथ इस्तेमाल करने वाला एक सिस्टम डिज़ाइन करें।

13. How would you support multiple prototype variants (e.g., per-country configs)?

Wrong: "Add if/else branches on country." *Good:* A keyed registry. *Excellent:* Good + discusses invalidation. *Follow-up:* What happens if config changes while the registry is live?

हिंदी: आप कई prototype वेरिएंट्स कैसे सपोर्ट करेंगे? — ग़लत: "country पर if/else शाखाएँ जोड़ो।" — अच्छा: एक keyed registry। — उत्कृष्ट: अच्छा + invalidation पर चर्चा करता है। — फ़ॉलो-अप: अगर registry लाइव रहते हुए कॉन्फ़िग बदले तो क्या होता है?

14. Is cloning thread-safe? *Wrong:* Blanket yes/no. *Good:* Fine under PHP-FPM; needs care in long-running processes. *Excellent:* Good + explains *why*. *Follow-up:* How does this differ in Node.js?

हिंदी: क्या cloning थ्रेड-सेफ़ है? — ग़लत: सामान्य हाँ/नहीं। — अच्छा: PHP-FPM के तहत ठीक है; लंबे समय तक चलने वाली प्रोसेसेज़ में सावधानी चाहिए। — उत्कृष्ट: अच्छा + क्यों समझाता है। — फ़ॉलो-अप: Node.js में यह कैसे अलग है?

15. Does Laravel use this pattern? *Wrong:* Confidently claims Eloquent "clones" models. *Good:* Names `replicate()` as the analogue. *Excellent:* Good + correctly states `replicate()` uses `new static()` + manual attribute filtering, not `clone`. *Follow-up:* What does `replicate()` exclude by default, and why?

हिंदी: क्या Laravel इस पैटर्न का इस्तेमाल करता है? — ग़लत: भरोसे से दावा करता है कि Eloquent मॉडल्स को "क्लोन" करता है। — अच्छा: `replicate()` को समकक्ष के तौर पर नाम देता है। — उत्कृष्ट: अच्छा + सही ढंग से कहता है कि `replicate()`, `clone` नहीं बल्कि `new static()` + मैनुअल attribute फ़िल्टरिंग इस्तेमाल करता है। — फ़ॉलो-अप: `replicate()` डिफ़ॉल्ट रूप से क्या बाहर रखता है, और क्यों?

Senior (5)

S1. Design a Prototype-based system for a specific expensive-construction scenario, and walk through your reasoning. *Excellent:* proactively raises invalidation, testing, and an anti-trigger case unprompted. *Follow-up:* What breaks first at 10x the load you designed for?

हिंदी: एक खास महँगे-निर्माण परिदृश्य के लिए एक Prototype-आधारित सिस्टम डिज़ाइन करें। — उत्कृष्ट: बिना पूछे invalidation, टेस्टिंग, और एक एंटी-ट्रिगर मामला सक्रिय रूप से उठाता है। — फ़ॉलो-अप: आपके डिज़ाइन किए गए लोड से 10 गुना पर पहले क्या टूटता है?

S2. A customer-reported data leak turns out to be a shallow-copy bug already in production. Walk through your response. *Excellent:* audits every other prototype class in the codebase for the same class of bug. *Follow-up:* How would this differ in a multi-tenant system?

हिंदी: एक ग्राहक-रिपोर्टेड डेटा लीक असल में प्रोडक्शन में पहले से मौजूद एक shallow-copy बग निकलती है। अपनी प्रतिक्रिया से गुजरें। — उत्कृष्ट: कोडबेस में बग की उसी श्रेणी के लिए हर दूसरी prototype क्लास का ऑडिट करता है। — फ़ॉलो-अप: एक मल्टी-टेनेंट सिस्टम में यह कैसे अलग होगा?

S3. When would you explicitly choose not to use Prototype even though it technically fits? *Excellent:* the deeply-nested-graph case, proposing immutability as the structural alternative. *Follow-up:* What would you tell a teammate who insists on using it for a session object anyway?

हिंदी: आप कब खुलकर Prototype इस्तेमाल न करने का चुनाव करेंगे भले ही यह तकनीकी रूप से फ़िट बैठे? — उत्कृष्ट: गहरे-nested-ग्राफ़ मामला, संरचनात्मक विकल्प के तौर पर immutability प्रस्तावित करता है। — फ़ॉलो-अप: आप एक साथी को क्या कहेंगे जो फिर भी इसे session ऑब्जेक्ट के लिए इस्तेमाल करने पर अड़ा है?

S4. How would you make deep-copy correctness structurally enforced rather than relying on developer discipline? *Excellent:* proposes immutability by default as the fix that removes the need for discipline entirely. *Follow-up:* Could static analysis catch a missing deep-copy?

हिंदी: आप deep-copy सटीकता को डेवलपर अनुशासन पर निर्भर रहने के बजाय संरचनात्मक रूप से कैसे लागू करेंगे? — उत्कृष्ट: डिफ़ॉल्ट रूप से immutability को उस फ़िक्स के तौर पर प्रस्तावित करता है जो अनुशासन की ज़रूरत को पूरी तरह हटा दे। — फ़ॉलो-अप: क्या स्टैटिक एनालिसिस एक गायब deep-copy पकड़ सकता है?

S5. A junior engineer proposes cloning a `PaymentTransaction` to quickly retry a failed payment. Response? *Excellent:* proposes constructing a fresh transaction that references the failed one's ID for audit purposes instead. *Follow-up:* What regulatory concerns apply in a fintech context?

हिंदी: एक शुरुआती इंजीनियर एक विफल पेमेंट को जल्दी दोबारा कोशिश करने के लिए एक `PaymentTransaction` क्लोन करने का प्रस्ताव रखता है। जवाब? — उत्कृष्ट: इसके बजाय एक ताज़ा transaction बनाने का प्रस्ताव रखता है जो audit उद्देश्यों के लिए विफल वाले की ID को संदर्भित करे। — फ़ॉलो-अप: fintech संदर्भ में कौन-सी नियामक चिंताएँ लागू होती हैं?

Staff (4)

ST1. How would you decide whether to standardize Prototype usage across multiple teams? *Excellent:* weighs the coupling cost of a shared abstraction against the correctness benefit. *Follow-up:* How would you roll it out without a big-bang migration?

हिंदी: आप कैसे तय करेंगे कि कई टीमों में Prototype इस्तेमाल को मानकीकृत करना है या नहीं? — उत्कृष्ट: साझा ऐब्स्ट्रैक्शन की कपलिंग लागत को सटीकता फ़ायदे के मुकाबले तौलता है। — फ़ॉलो-अप: आप इसे बिना बड़े-धमाके वाले माइग्रेशन के कैसे रोलआउट करेंगे?

ST2. How would you detect latent shallow-copy bugs across a large microservices fleet without manually auditing every codebase? *Excellent:* pairs a linter rule with a shared test-helper trait. *Follow-up:* CI-blocking or advisory?

हिंदी: आप हर कोडबेस को मैनुअल रूप से ऑडिट किए बिना एक बड़े माइक्रोसर्विसेज़ फ़्लीट में छिपी shallow-copy बग्स कैसे पकड़ेंगे? — उत्कृष्ट: एक linter नियम को एक साझा टेस्ट-हेल्पर trait के साथ जोड़ता है। — फ़ॉलो-अप: CI-ब्लॉकिंग या सलाहकारी?

ST3. Would you build a general-purpose `PrototypeRegistry` as shared platform infrastructure, or let each service own its own? *Excellent:* proposes a concrete decision rule. *Follow-up:* Who owns the on-call burden for a centrally-built registry?

हिंदी: क्या आप एक सामान्य-प्रयोजन `PrototypeRegistry` को साझा प्लेटफ़ॉर्म इन्फ़्रास्ट्रक्चर के तौर पर बनाएँगे, या हर सेवा को अपना बनाने देंगे? — उत्कृष्ट: एक ठोस निर्णय नियम प्रस्तावित करता है। — फ़ॉलो-अप: केंद्रीय रूप से बनी registry के लिए ऑन-कॉल बोझ का मालिक कौन है?

ST4. How does a region-keyed prototype registry interact with data-residency requirements in a regulated market? *Excellent:* flags the risk of a centrally-built registry inadvertently routing construction through infrastructure in the wrong jurisdiction. *Follow-up:* How would you audit this after the fact?

हिंदी: एक region-keyed prototype registry एक नियमित बाज़ार में डेटा-निवास ज़रूरतों के साथ कैसे इंटरैक्ट करती है? — उत्कृष्ट: उस जोखिम को चिह्नित करता है कि एक केंद्रीय रूप से बनी registry गलती से ग़लत क्षेत्राधिकार में निर्माण को रूट कर दे। — फ़ॉलो-अप: आप बाद में इसका ऑडिट कैसे करेंगे?

Principal (3)

P1. Describe a case where a correctly-chosen pattern still caused a production incident. What does that teach about pattern usage broadly? *Excellent:* generalizes: a pattern name describes intent, not a correctness guarantee. *Follow-up:* How do you instill that in engineers eager to apply patterns?

हिंदी: एक ऐसा मामला बताएँ जहाँ सही ढंग से चुना गया पैटर्न फिर भी एक प्रोडक्शन घटना का कारण बना। यह पैटर्न इस्तेमाल के बारे में व्यापक रूप से क्या सिखाता है? — उत्कृष्ट: सामान्यीकृत करता है: एक पैटर्न नाम इरादा बताता है, कोई सटीकता गारंटी नहीं। — फ़ॉलो-अप: आप उत्सुक इंजीनियरों में यह कैसे भरते हैं?

P2. Design review: a team wants to clone a `User` aggregate, including a mutable nested `Permissions` object, for a "duplicate user for testing" admin feature. Your feedback? *Excellent:* questions whether cloning is the right feature shape at all, proposing a "create from template with freshly-assigned permissions" flow instead. *Follow-up:* How do you phase this feedback so the team doesn't feel blocked?

हिंदी: डिज़ाइन रिव्यू: एक टीम एक "टेस्टिंग के लिए यूज़र डुप्लिकेट करो" एडमिन फ़ीचर के लिए एक `User` aggregate क्लोन करना चाहती है। आपकी प्रतिक्रिया? — उत्कृष्ट: सवाल करता है कि क्या cloning बिल्कुल सही फ़ीचर आकार है, इसके बजाय एक "ताज़ा-असाइन की गई permissions के साथ टेम्पलेट से बनाओ" फ़्लो प्रस्तावित करता है। — फ़ॉलो-अप: आप यह फ़ीडबैक कैसे चरणबद्ध करते हैं ताकि टीम को अवरुद्ध महसूस न हो?

P3. Summarize this entire pattern, for a new-hire principal engineer, in under two minutes. *Excellent:* matches the 1-minute pitch in Part 4 almost exactly. *Follow-up:* None — capstone question.

हिंदी: इस पूरे पैटर्न को, एक नए-नियुक्त प्रिंसिपल इंजीनियर के लिए, दो मिनट से कम में संक्षेप में बताएँ। — उत्कृष्ट: Part 4 की 1-मिनट पिच से लगभग बिल्कुल मेल खाता है। — फ़ॉलो-अप: कोई नहीं — समापन सवाल।

Coding problems (starter code + solutions in `Prototype.php`): (1) "Fix the Leak" — given a buggy `Customer / Address` pair, find and fix the shallow-copy bug. (2) "Build a Registry" — implement a keyed registry that never leaks the stored master. (3) "Deep Clone a Three-Level Graph" — `Order → Customer → Address`.

कोडिंग समस्याएँ (starter कोड + हल `Prototype.php` में): (1) "Fix the Leak" — एक बग वाली `Customer / Address` जोड़ी दी गई, shallow-copy बग ढूँढ़ें और ठीक करें। (2) "Build a Registry" — एक keyed registry इम्प्लीमेंट करें जो कभी संग्रहीत मास्टर को लीक न करे। (3) "Deep Clone a Three-Level Graph" — `Order → Customer → Address`।

✓ **Before you move on:** (1) Which single question above is most likely to actually come up as a follow-up inside an unrelated LLD round? (2) Can you deliver the Part 4 one-minute pitch from memory right now, unprompted?

✓ आगे बढ़ने से पहले: (1) ऊपर से कौन-सा एक सवाल किसी असंबंधित LLD राउंड में फॉलो-अप के तौर पर असल में आने की सबसे ज़्यादा संभावना है? (2) क्या आप अभी, बिना पूछे, याद से Part 4 की एक-मिनट पिच दे सकते हैं?

▢ APPENDIX

Part 22 — Learning Roadmap & Self-Assessment

Roadmap: the PHP manual's `clone / __clone` pages (primary source for exact semantics); the original GoF book's Prototype chapter; Laravel's `Illuminate\Database\Eloquent\Model::replicate()` source directly (verified in Part 11); re-implement this handbook's Tier 2 and Tier 3 examples from memory, then diff against `Prototype.php`.

रोडमैप: PHP मैनुअल के `clone / __clone` पेजेज़ (ठीक-ठीक अर्थशास्त्र के लिए प्राथमिक स्रोत); मूल GoF पुस्तक का Prototype अध्याय; Laravel के `Illuminate\Database\Eloquent\Model::replicate()` स्रोत को सीधे (Part 11 में सत्यापित); इस हैंडबुक के Tier 2 और Tier 3 उदाहरणों को याद से फिर से इम्प्लीमेंट करें, फिर `Prototype.php` से तुलना करें।

MCQs

1. What does `clone` call by default? (A: *nothing beyond the bitwise copy, unless `__clone()` is defined*)

हिंदी: `clone` डिफ़ॉल्ट रूप से क्या कॉल करता है? (उत्तर: *bitwise* कॉपी से आगे कुछ नहीं, जब तक `__clone()` डिफ़ाइन न हो)

1. Which property type is safe to leave shallow-copied with zero risk? (A: *an immutable/readonly value object*)

हिंदी: कौन-सी प्रॉपर्टी टाइप को बिना किसी जोखिम के shallow-copied छोड़ना सुरक्षित है? (उत्तर: एक अपरिवर्तनीय/readonly value ऑब्जेक्ट)

1. What should `PrototypeRegistry::get()` return? (A: *a clone, never the stored master*)

हिंदी: `PrototypeRegistry::get()` को क्या लौटाना चाहिए? (उत्तर: एक क्लोन, कभी संग्रहीत मास्टर नहीं)

1. Which PHP deployment model makes an incomplete `__clone()` a concurrency risk? (A: *long-running processes — Swoole/RoadRunner/persistent workers*)

हिंदी: कौन-सा PHP डिप्लॉयमेंट मॉडल एक अधूरे `__clone()` को concurrency जोखिम बनाता है? (उत्तर: लंबे समय तक चलने वाली प्रोसेसेज़ — Swoole/RoadRunner/स्थायी वर्कर्स)

1. Does Laravel's `replicate()` use `clone`? (A: *No — `new static()` + manual attribute filtering*)

हिंदी: क्या Laravel का `replicate()`, `clone` इस्तेमाल करता है? (उत्तर: नहीं — `new static()` + मैनुअल attribute फ़िल्टरिंग)

Scenario questions: - A notification service clones a per-locale template containing a mutable `array $placeholders` holding objects. Under load, placeholder values leak across notifications. Diagnose and fix. → Even though it's an array (value-copied in PHP), objects inside it are still shared references after a shallow clone — deep-copy each object inside the array within `__clone()`. - A stakeholder asks why you can't just cache the fully-rendered

invoice instead of using this pattern. Explain the distinction. → Full-result caching only works if the entire output is reusable as-is; here every invoice's order-specific data is genuinely unique.

परिदृश्य सवाल: - एक नोटिफिकेशन सेवा एक प्रति-locale टेम्पलेट क्लोन करती है जिसमें ऑब्जेक्ट्स रखने वाला एक परिवर्तनशील array \$placeholders है। लोड के तहत, placeholder मान नोटिफिकेशन्स में लीक होते हैं। निदान करें और ठीक करें। → भले ही यह एक array है (PHP में वैल्यू-कॉपीड), इसके अंदर ऑब्जेक्ट्स एक shallow clone के बाद भी साझा रेफरेंस हैं — `__clone()` के अंदर array के हर ऑब्जेक्ट को deep-copy करें। - एक स्टैकहोल्डर पूछता है कि आप इस पैटर्न का इस्तेमाल करने के बजाय पूरी तरह-रेंडर्ड invoice को सीधे कैश क्यों नहीं कर सकते। भेद समझाएँ। → पूर्ण-नतीजा कैशिंग तभी काम करती है जब पूरा आउटपुट जैसा-है वैसा पुनः इस्तेमाल-योग्य हो; यहाँ हर invoice का ऑर्डर-विशिष्ट डेटा सचमुच अद्वितीय है।

Refactoring exercise: given a `ReportGenerator` whose constructor loads chart-styling config, a logo, and a data-source connection, then accepts report-specific parameters — refactor into a correctly-implemented Prototype following the Part 19 journey, writing the clone-independence test first.

रीफैक्टरिंग अभ्यास: एक `ReportGenerator` दिया गया जिसका कंस्ट्रक्टर chart-styling कॉन्फिग, एक लोगो, और एक data-source कनेक्शन लोड करता है — Part 19 की यात्रा को फॉलो करते हुए एक सही ढंग से इम्प्लीमेंट किए गए Prototype में रीफैक्टर करें, पहले clone-independence टेस्ट लिखते हुए।

Architecture/debugging scenario: a `PrototypeRegistry` deployed as a long-running RoadRunner process shared across regions intermittently shows one region's currency symbol on another region's invoice under high concurrent load. Using Parts 9, 15, and 18: this is the shallow-copy leak escalated by the long-running-process concurrency framing — reproduce with a concurrent identity-check test, identify the incompletely-cloned property, fix with a cascading `__clone()`, add a permanent concurrent-access regression test.

आर्किटेक्चर/डीबगिंग परिदृश्य: क्षेत्रों में साझा एक लंबे समय तक चलने वाली RoadRunner प्रोसेस के तौर पर तैनात एक `PrototypeRegistry`, उच्च समवर्ती लोड के तहत रुक-रुक कर एक क्षेत्र का करेंसी चिह्न दूसरे क्षेत्र के invoice पर दिखाती है। Parts 9, 15, और 18 का इस्तेमाल करते हुए: यह लंबे-समय-तक-चलने-वाली-प्रोसेस concurrency फ्रेमिंग से बढ़ी हुई shallow-copy लीक है — एक समवर्ती पहचान-जाँच टेस्ट से दोहराएँ, अथूरी तरह क्लोन की गई प्रॉपर्टी पहचानें, एक कैस्केडिंग `__clone()` से ठीक करें, एक स्थायी समवर्ती-एक्सेस रिग्रेशन टेस्ट जोड़ें।

Technical Words Glossary / तकनीकी शब्दों की शब्दावली

English Term	Hindi Translation / हिंदी अनुवाद	Example / उदाहरण
Creational Pattern	क्रिएशनल पैटर्न	Prototype, Factory Method, और Singleton तीनों क्रिएशनल पैटर्न हैं।
Shallow Copy	शैलो (shallow) कॉपी	PHP का <code>clone</code> डिफ़ॉल्ट रूप से एक shallow कॉपी करता है।
Deep Copy	डीप (deep) कॉपी	nested ऑब्जेक्ट्स को स्वतंत्र रखने के लिए <code>__clone()</code> में एक deep कॉपी चाहिए।
Object Graph	ऑब्जेक्ट ग्राफ़	एक multi-level ऑब्जेक्ट ग्राफ़ में हर स्तर को अपना <code>__clone()</code> कैस्केड करना चाहिए।
Reference-counted Handle	रेफ़रेंस-काउंटेड हैंडल	PHP ऑब्जेक्ट्स को reference-counted हैंडल्स के ज़रिए एक्सेस किया जाता है।
Copy-on-write (COW)	कॉपी-ऑन-राइट	PHP में copy-on-write arrays पर लागू होता है, ऑब्जेक्ट ग्राफ़ पर नहीं।
Prototype Registry	प्रोटोटाइप रजिस्ट्री	एक keyed Prototype Registry कभी संग्रहीत मास्टर नहीं लौटाती, हमेशा एक क्लोन।
Immutability	अपरिवर्तनीयता (immutability)	अपरिवर्तनीयता deep-copy लॉजिक की पूरी ज़रूरत हटा देती है।
Identity Check (<code>assertNotSame</code>)	पहचान जाँच	<code>assertNotSame</code> clone-independence को साबित करता है, वैल्यू जाँच नहीं।
Event-driven Invalidation	इवेंट-ड्रिवन इनवैलिडेशन	एक <code>TaxRulesUpdated</code> इवेंट पर registry इवेंट-ड्रिवन इनवैलिडेशन इस्तेमाल करती है।
Value Object	वैल्यू ऑब्जेक्ट	एक अपरिवर्तनीय Value Object को साझा करना या क्लोन करना हमेशा सुरक्षित है।

General Words Glossary / सामान्य शब्दों की शब्दावली

English Word	Hindi Meaning / हिंदी अर्थ	Example / उदाहरण
Brevity	संक्षिप्तता	"Don't mistake the brevity for incompleteness." संक्षिप्तता को अधूरापन न समझें।
Calibrated	कैलिब्रेटेड, समायोजित	"This handbook is calibrated to how much interview time it actually earns." यह हैंडबुक इस पर कैलिब्रेटेड है कि यह असल में कितना इंटरव्यू समय कमाता है।
Fumble (verb)	लड़खड़ाना, चूक जाना	"Most candidates fumble this specific follow-up." ज्यादातर उम्मीदवार इस खास फ़ॉलो-अप में लड़खड़ाते हैं।
Papers over (idiom)	ढाँकना, सतही तौर पर छिपाना	"Hand-rolled caching just papers over the structural cause." हाथ से बनाई कैशिंग सिर्फ़ संरचनात्मक कारण को ढाँकती है।
Bolt on (idiom)	जोड़ देना, चिपका देना	"Don't bolt a second hierarchy onto the existing one awkwardly." मौजूदा पदानुक्रम पर एक दूसरे को अजीब तरीक़े से मत जोड़ें।
Trivially	तुच्छ रूप से, आसानी से	"The object is trivially cheap to construct." ऑब्जेक्ट बनाना तुच्छ रूप से सस्ता है।
Rehearsal	रिहर्सल, पूर्वाभ्यास	"This is a rehearsal scaffold, not a script." यह एक रिहर्सल ढाँचा है, कोई स्क्रिप्ट नहीं।
Sustained (adjective)	लगातार, बना रहने वाला	"Order volume spiked for a sustained window." ऑर्डर वॉल्यूम एक लगातार बनी रहने वाली विंडो के लिए बढ़ा।
Intermittent	रुक-रुक कर होने वाला	"It surfaced as an intermittent, hard-to-reproduce bug." यह एक रुक-रुक कर होने वाली, दोहराने में मुश्किल बग के तौर पर सामने आया।
Structural (fix)	संरचनात्मक	"Immutability is the structural fix, not just discipline." अपरिवर्तनीयता संरचनात्मक फ़िक्स है, सिर्फ़ अनुशासन नहीं।
Honest (bottom line)	ईमानदार	"The honest bottom line is that this pattern wasn't confirmed anywhere." ईमानदार निचोड़ यह है कि यह पैटर्न कहीं भी पुष्ट नहीं हुआ।
Reinvest (verb)	दोबारा निवेश करना	"Reinvest the prep time you saved here into the Very High patterns." यहाँ बचाया गया तैयारी समय Very High पैटर्न्स में दोबारा निवेश करें।

This handbook intentionally ends here rather than padding further — per Part 2, Prototype is a Low-frequency pattern across your five target markets. Reinvest the prep time you saved here into Strategy, Factory, Observer, and Singleton.

यह हैंडबुक जानबूझकर यहीं समाप्त होती है, आगे पैडिंग करने के बजाय — *Part 2* के अनुसार, *Prototype* आपके पाँच लक्षित बाज़ारों में एक कम-फ़्रीक्वेंसी पैटर्न है। यहाँ बचाया गया तैयारी समय *Strategy*, *Factory*, *Observer*, और *Singleton* में फिर से लगाएँ।

Code file (`Prototype.php`) is English-only; this handbook is bilingual English + Hindi throughout.

कोड फ़ाइल (`Prototype.php`) सिर्फ़ अंग्रेज़ी में है; यह हैंडबुक पूरी तरह अंग्रेज़ी + हिंदी द्विभाषी है।